Laporan Tugas 2

Kelompok 16

II4021 - Kriptografi

Penulis:

Luckman Fakhmanidris Arvasirri	18223041
Aldoy Fauzan Avanza	18223113
M Rabbani Khaizuran Arsyad	18223130

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

2026

DAFTAR ISI

DAFTAR ISI **2**
DAFTAR GAMBAR **4**
Teori Singkat **5**
Steganografi 5
Least Significant Bit (LSB) 5
Stream Cipher 6
Algoritma A5/1 6
Format dan Steganografi pada AVI 7
Format dan Steganografi pada MP4 7
Implementasi yang Dibebaskan 9
Transformasi Stego-key Menjadi Seed 9
Penyimpanan Metadata untuk Ekstraksi 9
Pemilihan Frame untuk Penyisipan 10
Perancangan dan Implementasi **11**
Bit Utility 11
A51 16
LSB 20
MP4 Container 30
GUI 34
histogram_window 34
extract_tab 36
embed_tab 43
app 54
Pengujian Program dan Analisis Hasil **56**
1. Kombinasi Konfigurasi (AVI) 56
Gambar 3.1 Capacity dan payload pengujian bagian 1 56
1.1. Konfigurasi 1.1 56
1.2. Konfigurasi 1.2 57
1.3. Konfigurasi 1.3 57
1.4. Konfigurasi 1.4 58
1.5. Konfigurasi 1.5 58
1.6. Konfigurasi 1.6 59
1.7. Konfigurasi 1.7 59
1.8. Konfigurasi 1.8 60
1.9. Konfigurasi 1.9 60
1.10. Konfigurasi 1.10 61
1.11. Konfigurasi 1.11 61
1.12. Konfigurasi 1.12 62
1.13. Analisis Hasil 62

Gambar 3.2 Filesize setiap hasil steganografi..... 62
2. Melebihi Kapasitas Sisip.....64
2.1. Konfigurasi 2.1.....64
2.2. Konfigurasi 2.2.....64
3. Berbagai Jenis Berkas.....65
3.1. Konfigurasi 3.1.....65
3.2. Konfigurasi 3.2.....65
3.3. Konfigurasi 3.3.....66
3.4. Konfigurasi 3.4.....66
3.5. Konfigurasi 3.5.....67
3.6. Analisis Hasil..... 67
4. Kombinasi Konfigurasi (MP4)..... 68
Gambar 3.3 Capacity dan payload pengujian bagian 4..... 68
4.1. Konfigurasi 4.1.....69
4.2. Konfigurasi 4.2.....69
4.3. Konfigurasi 4.3.....70
Kesimpulan..... 71
Daftar Pustaka.....73
Lampiran.....74

DAFTAR GAMBAR

Gambar 3.1 Capacity dan payload pengujian bagian 1 56
Gambar 3.2 Filesize setiap hasil steganografi 62
Gambar 3.3 Capacity dan payload pengujian bagian 4 68

Teori Singkat

Steganografi

Steganografi berasal dari bahasa Yunani, yaitu *steganos* yang berarti "tersembunyi" dan *graphein* yang berarti "menulis". Secara harfiah, steganografi dapat diartikan sebagai seni menulis pesan tersembunyi sedemikian rupa sehingga keberadaan pesan itu sendiri tidak dapat diketahui oleh pihak yang tidak berkepentingan. Berbeda dari kriptografi yang berfokus pada penyandian isi pesan sehingga tidak dapat dibaca meskipun diketahui keberadaannya, steganografi justru berfokus pada penyembunyian keberadaan pesan itu sendiri di dalam sebuah media pembawa (*cover object*). Dengan demikian, kedua teknik ini bersifat komplementer dan dalam implementasi modern sering digunakan secara bersamaan.

Media pembawa dalam steganografi dapat berupa berbagai jenis berkas digital, seperti citra (*image*), audio, video, hingga dokumen teks. Media tersebut dipilih berdasarkan kapasitas penyisipan yang dimiliki serta tingkat ketidakpekaan indera manusia terhadap perubahan kecil pada media tersebut. Hasil dari proses penyisipan disebut sebagai *stego-object*, sedangkan pesan yang disisipkan disebut sebagai *payload*. Terdapat tiga properti utama yang saling bertukar kepentingan (*trade-off*) dalam steganografi, yaitu kapasitas penyisipan, ketidakterdeksian (*imperceptibility*), dan ketahanan terhadap manipulasi (*robustness*).

Least Significant Bit (LSB)

Least Significant Bit (LSB) adalah salah satu metode steganografi paling mendasar dan paling banyak digunakan, khususnya pada media pembawa berupa citra dan video. Prinsip kerja metode ini memanfaatkan fakta bahwa perubahan pada bit-bit dengan nilai posisi paling rendah (*least significant*) dari suatu nilai piksel tidak akan menghasilkan perubahan visual yang dapat dideteksi oleh mata manusia secara kasat mata.

Setiap piksel pada citra berwarna (RGB) terdiri dari tiga komponen warna, yaitu merah (*Red*), hijau (*Green*), dan biru (*Blue*), yang masing-masing direpresentasikan dalam 8 bit (nilai 0–255). Dalam metode LSB, bit-bit dari pesan rahasia disisipkan ke dalam bit-bit paling tidak signifikan dari setiap komponen warna piksel. Karena nilai yang diubah hanya berskala kecil (misalnya nilai 200 menjadi 201), perubahan warna yang dihasilkan tidak terlihat oleh mata manusia.

Sebagai contoh, skema LSB **3-3-2** yang digunakan dalam implementasi ini berarti setiap 8 bit pesan dibagi ke dalam tiga bagian: 3 bit disisipkan ke dalam LSB komponen R, 3 bit ke dalam komponen G, dan 2 bit ke dalam komponen B. Pemilihan distribusi ini didasarkan pada sensitivitas mata manusia yang relatif lebih rendah terhadap perubahan pada komponen biru dibandingkan merah dan hijau, sehingga komponen B hanya menerima 2 bit modifikasi. Skema-skema alternatif seperti **4-2-2** dan **2-3-3** juga

didukung, yang menawarkan distribusi berbeda dengan karakteristik visual dan kapasitas yang sedikit berbeda pula.

Stream Cipher

Stream cipher adalah salah satu jenis algoritma enkripsi simetris yang mengenkripsi data secara bit per bit atau byte per byte, berbeda dengan *block cipher* yang memproses data dalam blok-blok berukuran tetap. Prinsip kerja *stream cipher* adalah dengan membangkitkan sebuah aliran bit semu-acak (*pseudorandom keystream*) yang kemudian di-XOR-kan dengan *plaintext* untuk menghasilkan *ciphertext*. Karena operasi XOR bersifat involusi, proses dekripsi dilakukan dengan cara yang persis sama: men-XOR-kan *ciphertext* dengan *keystream* yang identik untuk memperoleh kembali *plaintext*.

Keunggulan utama *stream cipher* dibandingkan *block cipher* adalah kecepatannya yang tinggi dan kemampuannya memproses data dengan panjang sembarang tanpa perlu *padding*. Hal ini menjadikan *stream cipher* sangat cocok untuk aplikasi steganografi, di mana ukuran pesan bervariasi dan kecepatan pemrosesan menjadi pertimbangan penting.

Algoritma A5/1

A5/1 adalah algoritma *stream cipher* yang dirancang untuk digunakan pada sistem komunikasi seluler GSM (*Global System for Mobile Communications*) sebagai mekanisme enkripsi jalur udara antara handset dan *base station*. Algoritma ini pertama kali digunakan pada tahun 1987 dan menjadi salah satu *stream cipher* yang paling banyak dipelajari dalam kriptanalisis modern.

A5/1 bekerja berdasarkan tiga buah *Linear Feedback Shift Register* (LFSR) yang saling berinteraksi melalui mekanisme *majority clocking*. Ketiga LFSR tersebut memiliki panjang yang berbeda-beda, yaitu:

- **R1**: panjang 19 bit, dengan polinomial *feedback* pada posisi bit ke-13, 16, 17, dan 18
- **R2**: panjang 22 bit, dengan polinomial *feedback* pada posisi bit ke-20 dan 21
- **R3**: panjang 23 bit, dengan polinomial *feedback* pada posisi bit ke-7, 20, 21, dan 22

Mekanisme *majority clocking* bekerja sebagai berikut: setiap siklus, diambil satu bit dari masing-masing register sebagai *clock bit* (bit ke-8 dari R1, bit ke-10 dari R2, dan bit ke-10 dari R3). Register hanya bergeser (*clock*) apabila nilai *clock bit*-nya sama dengan nilai mayoritas dari ketiga *clock bit* tersebut. Mekanisme ini menjadikan pergerakan ketiga register tidak sinkron, yang merupakan sumber utama kompleksitas kriptografi A5/1.

Proses inisialisasi A5/1 memerlukan dua buah masukan, yaitu kunci sesi (*session key*) sepanjang 64 bit dan nomor *frame* sepanjang 22 bit. Pada tahap *key loading*, seluruh bit

kunci dimasukkan satu per satu ke dalam ketiga register dengan cara meng-XOR-kan setiap bit ke dalam bit pertama masing-masing register, sambil tetap menggerakan seluruh register tanpa aturan *majority*. Proses yang sama dilakukan untuk *frame* pada tahap *frame loading*. Setelah itu, register dijalankan sebanyak 100 siklus tanpa menghasilkan keluaran sebagai tahap pemanasan (*warmup*).

Dalam implementasi ini, sesuai dengan ketentuan tugas, *payload* dibagi ke dalam blok-blok 228 bit dan nilai *frame* dibangkitkan secara otomatis dan berurutan (0, 1, 2, ...) untuk setiap blok. Kunci A5/1 berasal dari masukan pengguna yang dikodekan sebagai UTF-8 dan di-*pad* atau dipotong menjadi tepat 8 byte (64 bit). Karena enkripsi A5/1 bersifat simetris (XOR), fungsi dekripsi identik dengan fungsi enkripsi menggunakan kunci dan urutan *frame* yang sama.

Format dan Steganografi pada AVI

AVI (*Audio Video Interleave*) adalah format berkas video yang dikembangkan oleh Microsoft sebagai bagian dari teknologi *Video for Windows*. Berbeda dari format video modern yang umumnya menggunakan kompresi lossy, berkas AVI dapat menyimpan data video dengan codec yang bersifat *lossless* atau bahkan tanpa kompresi sama sekali. Hal ini menjadikan AVI sebagai media pembawa yang ideal untuk steganografi berbasis LSB pada piksel, karena data yang disisipkan ke dalam bit-bit piksel akan tetap tersimpan secara utuh dan tidak diubah oleh proses kompresi.

Secara struktural, berkas AVI menggunakan format RIFF (*Resource Interchange File Format*) yang mengorganisasi data dalam hierarki *chunk*. Setiap *chunk* memiliki pengenal empat karakter (*FourCC*) dan ukuran. Berkas AVI yang diproses oleh OpenCV dengan codec XVID atau MJPEG menyimpan data video sebagai urutan *frame* yang dapat diakses dan dimodifikasi secara independen.

Dalam implementasi ini, penyisipan pesan pada AVI dilakukan langsung pada nilai piksel setiap *frame* dengan metode LSB. Seluruh *frame* video dibaca menggunakan cv2.VideoCapture, dimodifikasi sesuai skema LSB yang dipilih, kemudian ditulis kembali menggunakan cv2.VideoWriter. Karena operasi dilakukan pada level piksel sebelum proses penulisan, dan codec yang dipilih bersifat *lossless*, nilai bit yang disisipkan dijamin tersimpan dengan tepat.

Format dan Steganografi pada MP4

MP4 (*MPEG-4 Part 14*) adalah format multimedia digital berbasis container yang dirancang untuk menyimpan video, audio, subtitle, dan data lainnya dalam satu berkas. Berbeda dari AVI yang merupakan format sederhana dengan data video tersimpan secara linier, MP4 mengorganisasi datanya dalam hierarki *box* (juga disebut *atom*) yang masing-masing memiliki header 8 byte: 4 byte pertama menyatakan ukuran *box* dalam format *big-endian*, dan 4 byte berikutnya adalah kode tipe *box*.

Terdapat tiga *box* utama dalam berkas MP4:

- **ftyp** (*File Type Box*): berisi tanda tangan (*signature*) berkas yang mengidentifikasi tipe dan codec yang kompatibel. *Box* ini selalu berada di awal berkas dan tidak boleh dimodifikasi.

- **mdat** (*Media Data Box*): berisi data mentah (*raw*) audio dan video dalam bentuk terkompresi. *Box* ini umumnya merupakan bagian terbesar dari berkas MP4 dan menjadi ruang kerja utama steganografi container.

- **moov** (*Movie Box*): berisi seluruh metadata video, termasuk informasi durasi, skala waktu, deskripsi codec, serta tabel-tabel offset yang menunjuk ke lokasi data dalam mdat. *Box* ini tidak boleh dimodifikasi ukurannya karena tabel offset di dalamnya, khususnya dalam sub-*box* stco (*Chunk Offset Box*), berisi alamat absolut yang menunjuk langsung ke posisi data dalam berkas.

moov tidak menyimpan data video/audio mentah, melainkan hanya menyimpan referensi ke lokasi data tersebut di dalam mdat. Oleh karena itu, apapun perubahan yang menggeser posisi data dalam mdat akan merusak seluruh referensi offset dan membuat video tidak dapat diputar.

Ketika sebuah berkas MP4 dibuat menggunakan pustaka seperti OpenCV, setiap *frame* yang ditulis akan melalui proses kompresi codec (H.264, MPEG-4, atau sejenisnya) yang bersifat *lossy*. Proses kompresi ini bekerja pada domain frekuensi menggunakan *Discrete Cosine Transform* (DCT): nilai piksel diubah ke representasi frekuensi, koefisien frekuensi tinggi yang dianggap tidak terlihat oleh mata manusia dihilangkan (kuantisasi), lalu data dikodekan kembali. Ketika *frame* dibaca kembali, dekoder merekonstruksi nilai piksel dari representasi frekuensi tersebut, yang menghasilkan nilai piksel yang sedikit berbeda dari aslinya. Bit-bit LSB yang telah disisipkan sebelum penulisan akan berubah secara tidak terduga dalam proses ini, sehingga seluruh data steganografi menjadi tidak terbaca. Dengan demikian, pendekatan LSB berbasis piksel yang berhasil pada AVI tidak dapat diterapkan secara langsung pada MP4 tanpa mekanisme koreksi kesalahan yang kompleks, dan hal tersebut berada di luar cakupan metode LSB standar.

Solusi yang tepat adalah melakukan penyisipan pada level *container*, yaitu langsung memanipulasi byte-byte di dalam *box* mdat tanpa melalui proses re-encoding video. Pendekatan ini didasarkan pada teknik yang dikembangkan oleh Domínguez Gómez (2021) dalam proyek *videostego*, dan bekerja sebagai berikut.

Berkas MP4 dibaca seluruhnya sebagai urutan byte mentah. Box mdat diidentifikasi dengan cara menelusuri struktur *box* dari awal berkas hingga ditemukan *box* bertipe mdat dengan ukuran lebih dari 8 byte. Selanjutnya, konten mdat dibagi menjadi tiga bagian yang sama: sepertiga pertama dan sepertiga terakhir dibiarkan tidak tersentuh, sedangkan sepertiga tengah ditetapkan sebagai ruang kerja (*workspace*). Alasan penghindaran sepertiga awal dan akhir adalah karena area-area tersebut dapat mengandung data inisialisasi codec atau penanda akhir yang sensitif, yang jika dimodifikasi dapat menyebabkan video tidak dapat diputar pada beberapa pemutar.

Implementasi yang Dibebaskan

Transformasi Stego-key Menjadi Seed

Untuk mendukung mode penyisipan acak, dibutuhkan sebuah nilai *seed* deterministik yang dapat direproduksi dari stego-key yang diberikan pengguna. Transformasi yang digunakan adalah sebagai berikut: stego-key dalam bentuk string dikodekan sebagai byte UTF-8, kemudian dihitung nilai hash SHA-256-nya. Enam belas byte pertama dari hasil hash diinterpretasikan sebagai integer 128-bit yang kemudian digunakan sebagai *seed* untuk generator bilangan acak Python (random.Random(seed)).

Penggunaan SHA-256 dipilih karena sifatnya yang deterministik (input sama selalu menghasilkan output sama), distribusi output yang seragam, serta ketahanannya terhadap analisis statistik. Dengan pendekatan ini, pengguna yang mengetahui stego-key yang sama akan selalu mendapatkan urutan piksel yang identik, sehingga proses ekstraksi dapat dilakukan dengan tepat. Stego-key itu sendiri tidak disimpan dalam stego-video.

stego_key (string)
→ encode UTF-8
→ SHA-256 hash (32 bytes)
→ 16 bytes pertama → int (128-bit)
→ random.Random(seed)
→ shuffle(pixel_indices)

Penyimpanan Metadata untuk Ekstraksi

Agar proses ekstraksi dapat dilakukan secara benar tanpa informasi di luar stego-video (kecuali kunci), seluruh informasi yang diperlukan untuk membaca kembali *payload* disisipkan sebagai bagian dari data yang disematkan, mendahului *payload* itu sendiri. Metadata ini dikodekan dalam format biner terstruktur dan disisipkan menggunakan metode dan skema yang sama dengan *payload*, sehingga tidak memerlukan *channel* penyimpanan terpisah.

Struktur metadata header yang digunakan adalah sebagai berikut:

Byte ke-0	: versi format (0x01)
Byte ke-1	: flags
	bit 0 = jenis pesan (0=teks, 1=berkas)
	bit 1 = status enkripsi (0=tidak, 1=ya)
	bit 2 = mode penyisipan (0=sequential, 1=acak)
Byte ke-2	: kode skema LSB (mis. 0x32 untuk "3-3-2")
Byte ke-3..6	: panjang payload dalam byte (uint32 big-endian)
Byte ke-7..8	: panjang nama berkas dalam byte (uint16 big-endian)
Byte ke-9..9+N	: nama berkas asli dalam encoding UTF-8

Header ini disisipkan terlebih dahulu, diikuti oleh *payload* (yang telah dienkripsi jika enkripsi diaktifkan). Pada saat ekstraksi, header dibaca terlebih dahulu untuk mengetahui

skema, panjang, dan properti *payload*, baru kemudian *payload* dibaca dan didekripsi sesuai kebutuhan. Kunci enkripsi A5/1 dan stego-key tidak disimpan dalam header maupun di bagian manapun dari stego-video, melainkan harus dimasukkan kembali oleh pengguna pada saat ekstraksi.

Pemilihan Frame untuk Penyisipan

Dalam implementasi ini, penyisipan dilakukan secara berurutan pada seluruh frame video, dimulai dari frame pertama. Setiap frame diisi penuh terlebih dahulu sebelum penyisipan dilanjutkan ke frame berikutnya. Frame-frame yang tidak diperlukan untuk menampung seluruh data (karena payload sudah habis disisipkan sebelum frame terakhir) disalin tanpa modifikasi apapun. Strategi ini memaksimalkan kapasitas yang tersedia sekaligus meminimalkan jumlah frame yang mengalami degradasi kualitas.

Pertimbangan di balik pemilihan strategi ini adalah kesederhanaan implementasi dan efisiensi kapasitas. Pendekatan alternatif seperti penyisipan pada frame-frame bernomor genap saja, atau hanya pada frame yang memiliki tingkat detail visual tinggi (berdasarkan variansi piksel), memang dapat meningkatkan ketidakterdeksian, namun membutuhkan mekanisme koordinasi tambahan antara proses embed dan extract yang meningkatkan kompleksitas sistem secara keseluruhan.

Untuk mode penyisipan acak, urutan piksel dalam setiap frame diacak menggunakan generator bilangan acak berdasarkan seed yang diturunkan dari stego-key (sebagaimana dijelaskan pada 2.6.1). Urutan frame tetap berurutan dari frame pertama, yang diacak hanyalah urutan piksel dalam setiap frame. Hal ini memastikan bahwa distribusi data pesan tersebar secara acak di seluruh area frame, bukan terkonsentrasi di sudut kiri atas, sehingga meningkatkan ketahanan terhadap analisis statistik visual.

Perancangan dan Implementasi

Bit Utility

Utility function yang membantu perubahan tipe data, seperti dari file input (PDF, csv, txt) ke dalam bentuk bit untuk disisipkan kedalam AVI dengan tool stego. Membantu mengambil value RGB dari cover AVI untuk disisipkan. Berikut fungsi-fungsi yang dibuat

bytes_to_bits

```
def bytes_to_bits(data):
    """Convert bytes to a sequence of bits.

    Useful in LSB steganography for expanding payload
    bytes into a bit stream
    that can be embedded into image pixel channels.
    """
    bits = []
    for byte in data:
        for i in range(8):
            bits.append((byte >> (7 - i)) & 1)
    return bits
```

Mengubah data bytes menjadi bits.
Membantu penyisipan pesan kepada bit pixel RGB

bits_to_bytes

```
def bits_to_bytes(bits):
    """Convert a sequence of bits back into a bytes object.

    Used to reconstruct payload bytes from extracted bit
stream during decoding.
    Pads a partial final byte with zeros if necessary.
    """
    out = bytearray()
    for i in range(0, len(bits), 8):
        chunk = bits[i:i+8]
        if len(chunk) < 8:
```

```
            chunk = chunk + [0] * (8 - len(chunk))
        val = 0
        for b in chunk:
            val = (val << 1) | b
        out.append(val)
    return bytes(out)
```

Mengubah data bits menjadi bytes
Membantu penyusunan pesan yang disisipkan saat proses dekripsi

parse_scheme

```
def parse_scheme(scheme_str):
    """Parse a scheme string like '2-3-1' into integer R/G/B
bit allocations."""
    parts = scheme_str.split('-')
    return int(parts[0]), int(parts[1]), int(parts[2])
```

mengambil nilai red, green, blue dari schema RGB (r-g-b) dan mengembalikan nilainya dalam bentuk list of integer dengan urutan pertama red dilanjut dengan green dan terakhir blue

scheme_to_byte

```
def scheme_to_byte(scheme_str):
    """Encode a scheme string into a compact single byte for
header storage."""
    r, g, b = parse_scheme(scheme_str)
    return (r << 4) | (g << 2) | b
```

Mengubah nilai RGB yang ada dalam bentuk skema nilai RGB(r-g-b) menjadi bentuk skema byte (rrrrrrrr gggggggg bbbb) dengan bantuan fungsi parse_scheme

byte_to_scheme

```
 """Decode compact header scheme byte back into 'R-G-B'
string."""
    r = (val >> 4) & 0xF
    g = (val >> 2) & 0x3
    b = val & 0x3
    return f"{r}-{g}-{b}"
```

Mengubah nilai RGB skema byte menjadi skema r-g-b

pack_header

```python
 def pack_header(is_file, is_encrypted, is_random,
scheme_str, payload_len, filename):
    """Pack metadata header for payload embedding.

    Header contains version, flags, scheme, payload length,
filename length and filename bytes.
    """
    flags = 0
    if is_file: flags |= 0x01
    if is_encrypted: flags |= 0x02
    if is_random: flags |= 0x04

    scheme_byte = scheme_to_byte(scheme_str)
    fname_bytes = filename.encode('utf-8') if filename else
b''
    fname_len = len(fname_bytes)

    header = bytearray()
    header.append(0x01)  # version
    header.append(flags)
    header.append(scheme_byte)
    header += struct.pack('>I', payload_len)
    header += struct.pack('>H', fname_len)
    header += fname_bytes
    return bytes(header)
```

Membuat header berisi metadata pesan(payload)
Berisi versi, flag untuk memastikan payload adalah file, flag enkripsi dan flag bahwa payload menggunakan pixel random, skema RGB, panjang payload, panjang nama file, dan nama file dalam bentuk bytes

unpack_header_from_bits

```python
def unpack_header_from_bits(bits):
    """Unpack and validate header data from an extracted bit
stream.

    Returns metadata plus the number of header bits consumed
by the payload.
    """
    if len(bits) < 72:
```

```
        raise ValueError("Not enough data to read header")

    hdr_bytes = bits_to_bytes(bits[:72])
    version = hdr_bytes[0]
    if version != 0x01:
        raise ValueError(f"Unknown stego version: {version}")

    flags = hdr_bytes[1]
    is_file = bool(flags & 0x01)
    is_encrypted = bool(flags & 0x02)
    is_random = bool(flags & 0x04)

    scheme_byte = hdr_bytes[2]
    scheme_str = byte_to_scheme(scheme_byte)

    payload_len = struct.unpack('>I', hdr_bytes[3:7])[0]
    fname_len = struct.unpack('>H', hdr_bytes[7:9])[0]

    total_hdr_bytes = 9 + fname_len
    total_hdr_bits = total_hdr_bytes * 8

    if len(bits) < total_hdr_bits:
        raise ValueError("Not enough data for filename in
header")

    fname_raw = bits_to_bytes(bits[72:total_hdr_bits])
    filename = fname_raw[:fname_len].decode('utf-8',
errors='replace')

    return {
        'is_file': is_file, 'is_encrypted': is_encrypted,
        'is_random': is_random, 'scheme': scheme_str,
        'payload_len': payload_len, 'filename': filename,
        'header_bits': total_hdr_bits
    }
```

Membongkar header untuk memperoleh metadata dari payload dan panjan payload dalam bits

cals_mse_psnr

```python
"""Calculate MSE and PSNR between original and stego image
arrays.

    Used to report image quality degradation after LSB
embedding.
    """
    diff = orig.astype(np.float64) - stego.astype(np.float64)
    mse = np.mean(diff ** 2)
    if mse == 0:
        return 0.0, float('inf')
    psnr = 10 * np.log10(255**2 / mse)
    return mse, psnr
```

Fungsi untuk MSE(Mead Squared and Error) dan PSNR(Peak Signal-to-Noise Ratio) untuk menilai kualitas visual secara otomatis

sha256_digest

```python
def sha256_digest(data: bytes) -> str:
    return hashlib.sha256(data).hexdigest()
```

A51

Merupakan algoritma cipher pesan, sebelum pesan disisipkan kedalam cover AVI

Pengaturan A5/1 stream cipher

```
BLOCK_BITS = 228

# LFSR sizes
R1_LEN = 19
R2_LEN = 22
R3_LEN = 23

# Feedback tap
R1_TAPS = [13, 16, 17, 18]
R2_TAPS = [20, 21]
R3_TAPS = [7, 20, 21, 22]

# Clock bit
R1_CLK = 8
R2_CLK = 10
R3_CLK = 10
```

BLOCK_BITS mengatur jumlah bit per blok keystream (228 bit, sesuai burst GSM). Ukuran LFSR menentukan panjang state tiap register (R1=19, R2=22, R3=23; total 64 bit).
Feedback tap menentukan bit-bit yang di-XOR untuk menghasilkan feedback bit. Clock bit digunakan pada mekanisme majority clocking: hanya register dengan clock bit yang sama dengan mayoritas yang digeser(shifting).

_get_bit

```
def _get_bit(reg, pos):
    return (reg >> pos) & 1
```

Mengambil LSB dari hasil shifting register ke kanan dengan pos

_clock_reg

```
def _clock_reg(reg, taps, length):
    feedback = 0
    for t in taps:
```

```
        feedback ^= _get_bit(reg, t)
    reg = (reg >> 1) | (feedback << (length - 1))
    return reg
```

mengembalikan nilai register setelah 1 proses clock, yaitu register digeser ke kanan dan bit paling kiri diisi feedback hasil XOR dari tap.

_majority

```
def _majority(r1, r2, r3):
    b1 = _get_bit(r1, R1_CLK)
    b2 = _get_bit(r2, R2_CLK)
    b3 = _get_bit(r3, R3_CLK)
    return int(b1 + b2 + b3 >= 2)
```

memberikan nilai untuk menentukan clocking register, dipakai untuk clocking di fungsi _clock_all_majority

_clock_all_majority

```
def _clock_all_majority(r1, r2, r3):
    maj = _majority(r1, r2, r3)
    if _get_bit(r1, R1_CLK) == maj:
        r1 = _clock_reg(r1, R1_TAPS, R1_LEN)
    if _get_bit(r2, R2_CLK) == maj:
        r2 = _clock_reg(r2, R2_TAPS, R2_LEN)
    if _get_bit(r3, R3_CLK) == maj:
        r3 = _clock_reg(r3, R3_TAPS, R3_LEN)
    return r1, r2, r3
```

Fungsi ini menghitung nilai majority, lalu melakukan clocking hanya pada register yang clock bit-nya sama dengan nilai majority.

_init_registers

```
def _init_registers(kc_bits, fn_bits):
    r1 = r2 = r3 = 0

    # load key (64 bits)
    for i in range(64):
        r1 = _clock_reg(r1, R1_TAPS, R1_LEN)
        r2 = _clock_reg(r2, R2_TAPS, R2_LEN)
        r3 = _clock_reg(r3, R3_TAPS, R3_LEN)
```

```
        bit = kc_bits[i]
        r1 ^= bit
        r2 ^= bit
        r3 ^= bit

    # load frame number (22 bits)
    for i in range(22):
        r1 = _clock_reg(r1, R1_TAPS, R1_LEN)
        r2 = _clock_reg(r2, R2_TAPS, R2_LEN)
        r3 = _clock_reg(r3, R3_TAPS, R3_LEN)
        bit = fn_bits[i]
        r1 ^= bit
        r2 ^= bit
        r3 ^= bit

    for _ in range(100):
        r1, r2, r3 = _clock_all_majority(r1, r2, r3)

    return r1, r2, r3
```

Menginisialisasi ketiga LFSR dengan bit kunci dan bit nomor frame, lalu diclock 100 kali untuk mengacak state awal sebelum keystream dibangkitkan.

_generate_keystream

```
def _generate_keystream(kc_bits, fn, n_bits):
    fn_bits = [(fn >> i) & 1 for i in range(22)]
    r1, r2, r3 = _init_registers(kc_bits, fn_bits)

    stream = []
    for _ in range(n_bits):
        r1, r2, r3 = _clock_all_majority(r1, r2, r3)
        out = _get_bit(r1, R1_LEN-1) ^ _get_bit(r2, R2_LEN-1)
^ _get_bit(r3, R3_LEN-1)
        stream.append(out)
    return stream
def _generate_keystream(kc_bits, fn, n_bits):
    fn_bits = [(fn >> i) & 1 for i in range(22)]
    r1, r2, r3 = _init_registers(kc_bits, fn_bits)
```

```
    stream = []
    for _ in range(n_bits):
        r1, r2, r3 = _clock_all_majority(r1, r2, r3)
        out = _get_bit(r1, R1_LEN-1) ^ _get_bit(r2, R2_LEN-1)
^ _get_bit(r3, R3_LEN-1)
        stream.append(out)
    return stream
```

Membangkitkan keystream sepanjang n bit menggunakan ketiga LFSR yang sudah diinisialisasi. Setiap bit keystream dihasilkan dari XOR bit output ketiga LFSR setelah di-clock dengan majority clocking.

_key_to_bits

```
def _key_to_bits(key_str):
    raw = key_str.encode('utf-8')
    if len(raw) < 8:
        raw = raw + b'\x00' * (8 - len(raw))
    else:
        raw = raw[:8]
    bits = []
    for byte in raw:
        for i in range(8):
            bits.append((byte >> i) & 1)
    return bits
```

Mengubah kunci string menjadi 64 bit dengan cara mengkonversi ke bytes UTF-8 yang disesuaikan panjangnya menjadi tepat 8 byte, lalu diurai per bit dengan urutan LSB terlebih dahulu.

encrypt

```
def encrypt(plaintext_bytes, key_str):
    kc_bits = _key_to_bits(key_str)
    plain_bits = []
    for b in plaintext_bytes:
        for i in range(8):
            plain_bits.append((b >> (7 - i)) & 1)

    cipher_bits = []
```

```
    fn = 0
    for offset in range(0, len(plain_bits), BLOCK_BITS):
        chunk = plain_bits[offset:offset + BLOCK_BITS]
        ks = _generate_keystream(kc_bits, fn, len(chunk))
        for j in range(len(chunk)):
            cipher_bits.append(chunk[j] ^ ks[j])
        fn += 1

    out = bytearray()
    for i in range(0, len(cipher_bits), 8):
        byte_bits = cipher_bits[i:i+8]
        if len(byte_bits) < 8:
            byte_bits += [0] * (8 - len(byte_bits))
        val = 0
        for bit in byte_bits:
            val = (val << 1) | bit
        out.append(val)
    return bytes(out)
```

Mengenkripsi plaintext bytes dengan cara meng-XOR tiap blok bit plaintext dengan keystream yang dibangkitkan per nomor frame secara berurutan, lalu hasilnya disusun kembali menjadi bytes.

decrypt

```
def decrypt(ciphertext_bytes, key_str):
    return encrypt(ciphertext_bytes, key_str)
```

Mendekripsi ciphertext menggunakan fungsi encrypt

LSB

Merupakan function yang menyisipkan pesan kedalam cover AVI dengan metode LSB

get_capacity

```
def get_capacity(video_path, scheme="3-3-2"):
    r_bits, g_bits, b_bits = parse_scheme(scheme)
    bpp = r_bits + g_bits + b_bits
```

```
    cap = cv2.VideoCapture(video_path)
    if not cap.isOpened():
        raise IOError(f"Can't open video: {video_path}")

    total_pixels = 0
    while True:
        ret, frame = cap.read()
        if not ret:
            break
        h, w = frame.shape[:2]
        total_pixels += w * h
    cap.release()

    total_bits = total_pixels * bpp
    return total_bits // 8
def get_capacity(video_path, scheme="3-3-2"):
    r_bits, g_bits, b_bits = parse_scheme(scheme)
    bpp = r_bits + g_bits + b_bits

    cap = cv2.VideoCapture(video_path)
    if not cap.isOpened():
        raise IOError(f"Can't open video: {video_path}")

    total_pixels = 0
    while True:
        ret, frame = cap.read()
        if not ret:
            break
        h, w = frame.shape[:2]
        total_pixels += w * h
    cap.release()

    total_bits = total_pixels * bpp
    return total_bits // 8
```

Mengembalikan kapasitas data rahasia (dalam byte) yang bisa disisipkan ke video berdasarkan skema LSB, bukan ukuran asli file video dalam byte.

_get_pixel_order

```
def _get_pixel_order(w, h, use_random, stego_key, frame_idx):
    n_pixels = w * h
    indices = list(range(n_pixels))
    if use_random and stego_key:
        rng = _random.Random(f"{stego_key}_{frame_idx}")
        rng.shuffle(indices)
    return indices
```

Mengembalikan daftar urutan indeks pixel yang akan dipakai untuk proses embed/extract.
Jika use_random aktif dan stego_key ada, urutan pixel diacak secara deterministik berdasarkan kombinasi stego_key dan frame_idx

_embed_bits_in_frame

```
def _embed_bits_in_frame(frame, bits, offset, scheme,
use_random, stego_key, frame_idx):
    r_bits, g_bits, b_bits = parse_scheme(scheme)
    bpp = r_bits + g_bits + b_bits
    h, w = frame.shape[:2]
    order = _get_pixel_order(w, h, use_random, stego_key,
frame_idx)

    stego = frame.copy()
    idx = offset
    for px_i in order:
        if idx >= len(bits):
            break
        y = px_i // w
        x = px_i % w
        pixel = list(stego[y, x])

        channel_bits = [(2, r_bits), (1, g_bits), (0,
b_bits)]
        for ch, n in channel_bits:
            if n == 0 or idx >= len(bits):
                continue
            mask = (0xFF << n) & 0xFF
            val = pixel[ch] & mask
            for bit_pos in range(n - 1, -1, -1):
                if idx < len(bits):
```

```
                    val |= (bits[idx] << bit_pos)
                    idx += 1
            pixel[ch] = val

        stego[y, x] = pixel

    return stego, idx
```

Berfungsi menulis bit payload ke 1 frame video, lalu mengembalikan frame hasil stego dan posisi bit terakhir yang sudah ditulis.

_extract_bits_from_frame

```
def _extract_bits_from_frame(frame, n_bits, offset, scheme,
use_random, stego_key, frame_idx):
    r_bits, g_bits, b_bits = parse_scheme(scheme)
    bpp = r_bits + g_bits + b_bits
    h, w = frame.shape[:2]
    order = _get_pixel_order(w, h, use_random, stego_key,
frame_idx)

    extracted = []
    count = 0
    for px_i in order:
        if count >= n_bits:
            break
        y = px_i // w
        x = px_i % w
        pixel = frame[y, x]

        channel_bits = [(2, r_bits), (1, g_bits), (0,
b_bits)]
        for ch, n in channel_bits:
            if n == 0:
                continue
            for bit_pos in range(n - 1, -1, -1):
                if count < n_bits:
                    extracted.append((pixel[ch] >> bit_pos) &
1)
```

```
                    count += 1

    return extracted
```

Mengekstrak sejumlah bit payload (n_bits) dari satu frame stego berdasarkan skema bit per kanal RGB dan urutan piksel (normal atau acak), lalu mengembalikannya sebagai list bit.

_detect_format

```
def _detect_format(path):
    ext = os.path.splitext(path)[1].lower()
    if ext == '.mp4':
        return 'mp4'
    return 'avi'
```

Mengenali format dari file berdasarkan path yang diberikan

_get_writer

```
def _get_writer(out_path, fps, w, h):
    fmt = _detect_format(out_path)
    if fmt == 'mp4':
        fourcc = cv2.VideoWriter_fourcc(*'mp4v')
    else:
        fourcc = cv2.VideoWriter_fourcc(*'FFV1')
    writer = cv2.VideoWriter(out_path, fourcc, fps, (w, h))
    if not writer.isOpened() and fmt == 'avi':
        fourcc = cv2.VideoWriter_fourcc(*'XVID')
        writer = cv2.VideoWriter(out_path, fourcc, fps, (w,
h))
    return writer
```

Membuat dan mengembalikan objek VideoWriter untuk menyimpan video output, memilih codec berdasarkan format file output (mp4 atau avi), dan mencoba codec cadangan jika writer AVI gagal dibuka.

embed

```
def embed(video_path, out_path, payload_bytes, scheme,
is_file, filename,
          use_encryption, key, use_random, stego_key,
```

```
progress_cb=None):

    if use_encryption and key:
        enc_payload = a51_encrypt(payload_bytes, key)
    else:
        enc_payload = payload_bytes
        use_encryption = False

    header = pack_header(is_file, use_encryption, use_random,
scheme,
                         len(enc_payload), filename if
is_file else "")
    full_data = header + enc_payload
    all_bits = bytes_to_bits(full_data)

    cap_bytes = get_capacity(video_path, scheme)
    needed_bytes = len(full_data)
    if needed_bytes > cap_bytes:
        raise ValueError(f"Payload too large! Need
{needed_bytes} bytes but capacity is {cap_bytes} bytes")

    cap = cv2.VideoCapture(video_path)
    fps = cap.get(cv2.CAP_PROP_FPS) or 30
    w = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
    h = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

    writer = _get_writer(out_path, fps, w, h)

    mse_vals = []
    psnr_vals = []
    bit_idx = 0
    frame_num = 0

    while True:
        ret, frame = cap.read()
        if not ret:
            break
```

```
        if bit_idx < len(all_bits):
            stego_frame, bit_idx = _embed_bits_in_frame(
                frame, all_bits, bit_idx, scheme, use_random,
stego_key, frame_num)
            mse, psnr = calc_mse_psnr(frame, stego_frame)
            mse_vals.append(mse)
            psnr_vals.append(psnr)
            writer.write(stego_frame)
        else:
            writer.write(frame)

        frame_num += 1
        if progress_cb:
            progress_cb(frame_num, total_frames)

    cap.release()
    writer.release()

    avg_mse = np.mean(mse_vals) if mse_vals else 0
    avg_psnr = np.mean([p for p in psnr_vals if p !=
float('inf')]) if psnr_vals else float('inf')

    return avg_mse, avg_psnr, mse_vals, psnr_vals
```

Fungsi utama untuk menyisipkan payload ke video cover (AVI/MP4), termasuk opsi enkripsi, pembuatan header metadata, pengecekan kapasitas, proses embedding per frame, serta penyimpanan video stego.
Fungsi ini mengembalikan rata-rata MSE dan PSNR beserta daftar MSE/PSNR per frame.

extract

```
def extract(stego_path, a51_key=None, stego_key=None):
    cap = cv2.VideoCapture(stego_path)
    if not cap.isOpened():
        raise IOError(f"Can't open stego video:
{stego_path}")

    frames = []
    while True:
```

```
        ret, frame = cap.read()
        if not ret:
            break
        frames.append(frame)
    cap.release()

    if not frames:
        raise ValueError("No frames in stego video")

    SCHEMES = ["3-3-2", "4-2-2", "2-3-3"]
    header_info = None
    working_scheme = None
    PROBE_BITS = 2048

    for try_scheme in SCHEMES:
        try:
            hdr_bits = _extract_bits_from_frame(
                frames[0], PROBE_BITS, try_scheme, False,
None, 0)
            info = unpack_header_from_bits(hdr_bits)
            if info['scheme'] == try_scheme:
                header_info = info
                working_scheme = try_scheme
                break
        except:
            continue

    if header_info is None and stego_key:
        for try_scheme in SCHEMES:
            try:
                hdr_bits = _extract_bits_from_frame(
                    frames[0], PROBE_BITS, try_scheme, True,
stego_key, 0)
                info = unpack_header_from_bits(hdr_bits)
                if info['scheme'] == try_scheme:
                    header_info = info
                    working_scheme = try_scheme
                    break
            except:
```

```
                continue

    if header_info is None:
        raise ValueError("Failed to parse stego header: wrong
key or not a stego video")

    use_random = header_info['is_random']
    scheme = header_info['scheme']
    payload_len = header_info['payload_len']
    total_bits_needed = header_info['header_bits'] +
payload_len * 8

    all_extracted = []
    bits_remaining = total_bits_needed
    for fi, frame in enumerate(frames):
        if bits_remaining <= 0:
            break
        r_bits, g_bits, b_bits = parse_scheme(scheme)
        bpp = r_bits + g_bits + b_bits
        frame_capacity = frame.shape[0] * frame.shape[1] *
bpp
        n = min(bits_remaining, frame_capacity)
        frame_bits = _extract_bits_from_frame(
            frame, n, scheme, use_random, stego_key if
use_random else None, fi)
        all_extracted.extend(frame_bits)
        bits_remaining -= len(frame_bits)

    hdr = unpack_header_from_bits(all_extracted)
    payload_start = hdr['header_bits']
    payload_bits = all_extracted[payload_start:payload_start
+ payload_len * 8]
    payload_raw = bits_to_bytes(payload_bits)[:payload_len]

    if hdr['is_encrypted']:
        if not a51_key:
            raise ValueError("Payload is encrypted but no
A5/1 key provided")
        payload_raw = a51_decrypt(payload_raw, a51_key)
```

```
    return payload_raw, hdr['is_file'], hdr['filename']
```

Fungsi utama untuk mengekstrak payload dari video stego (MP4/AVI), membaca header metadata, menentukan skema/urutan ekstraksi, lalu mendekripsi payload jika diperlukan.
Fungsi mengembalikan payload_raw, status is_file, dan filename.

MP4 Container

find_mdat_block

```
def find_mdat_block(data: bytes) -> tuple:
    pos = 0
    while pos < len(data) - 8:
        box_size = int.from_bytes(data[pos:pos + 4], 'big')
        box_type = data[pos + 4:pos + 8]
        if box_type == b'mdat' and box_size > 8:
            return pos + 8, box_size - 8
        if box_size == 0 or box_size < 8:
            break
        pos += box_size
    raise ValueError("No suitable mdat block found in this MP4 file")
```

Mencari blok mdat dalam file MP4 dan mengembalikan posisi awal konten serta ukurannya.

_get_workspace

```
def _get_workspace(content_start: int, content_size: int) -> tuple:
    third = content_size // 3
    return content_start + third, content_start + 2 * third
```

Menentukan area kerja penyisipan pesan, yaitu sepertiga tengah dari blok mdat

get_capacity

```
def get_capacity(mp4_path: str) -> int:
    with open(mp4_path, 'rb') as f:
        data = f.read()
    content_start, content_size = find_mdat_block(data)
    ws_start, ws_end = _get_workspace(content_start, content_size)
    return max(0, (ws_end - ws_start) // 8 - 14)
```

Menghitung kapasitas maksimal pesan yang dapat disisipkan ke dalam file MP4 berdasarkan ukuran workspace yang tersedia.

_parity_encode

```
def _parity_encode(workspace: bytearray, offset: int, bits:
list) -> None:
    for i, bit in enumerate(bits):
```

```
        b = workspace[offset + i]
        if b % 2 != bit:
            workspace[offset + i] = b + 1 if b < 0xFF else b
- 1
```

Menyisipkan bit pesan ke dalam workspace dengan mengubah paritas tiap byte sesuai bit yang ingin disisipkan.

_parity_decode

```
def _parity_decode(data: bytes, offset: int, n_bits: int) ->
list:
    return [data[offset + i] % 2 for i in range(n_bits)]
```

Membaca bit pesan dari data dengan mengambil paritas tiap byte pada posisi yang ditentukan.

_build_embed_data

```
def _build_embed_data(payload: bytes, is_file: bool,
filename: str,
                      is_encrypted: bool, is_random: bool) ->
bytes:
    fname_bytes = filename.encode('utf-8') if filename else
b''
    flags = (0x01 if is_file else 0) | (0x02 if is_encrypted
else 0) | (0x04 if is_random else 0)

    inner = struct.pack('>BB', flags, 0x32)
    inner += struct.pack('>IH', len(payload),
len(fname_bytes))
    inner += fname_bytes + payload

    total_size = 2 + 4 + len(inner)
    return struct.pack('>H', total_size) + b'STEG' + inner
```

Menyusun data yang akan disisipkan menjadi satu struktur bytes lengkap, mencakup header, flag, ukuran payload, nama file, dan payload itu sendiri.

embed_mp4

```
def embed_mp4(src_path: str, out_path: str, payload: bytes,
              is_file: bool, filename: str,
```

```
                is_encrypted: bool, is_random: bool) -> None:
    with open(src_path, 'rb') as f:
        data = bytearray(f.read())

    content_start, content_size =
find_mdat_block(bytes(data))
    ws_start, ws_end = _get_workspace(content_start,
content_size)
    workspace_size = ws_end - ws_start

    bits = bytes_to_bits(_build_embed_data(payload, is_file,
filename, is_encrypted, is_random))

    if len(bits) > workspace_size:
        raise ValueError(
            f"Payload too large: need {len(bits)} workspace
bytes, "
            f"have {workspace_size} (max payload ≈
{(workspace_size // 8) - 14:,} bytes)"
        )

    _parity_encode(data, ws_start, bits)

    with open(out_path, 'wb') as f:
        f.write(data)
```

Menyisipkan pesan ke dalam file MP4 dengan cara meng-encode data ke workspace pada blok mdat menggunakan parity encode, lalu menyimpan hasilnya ke file baru.

extract_mp4

```
def extract_mp4(stego_path: str) -> tuple:
    with open(stego_path, 'rb') as f:
        data = f.read()

    content_start, content_size = find_mdat_block(data)
    ws_start, ws_end = _get_workspace(content_start,
content_size)
    workspace_size = ws_end - ws_start
```

```
    if workspace_size < 16:
        raise ValueError("mdat workspace too small to contain
stego header")

    total_size = struct.unpack('>H',
bits_to_bytes(_parity_decode(data, ws_start, 16)))[0]

    if total_size < 14 or total_size * 8 > workspace_size:
        raise ValueError("Failed to parse stego header: wrong
key or not a stego video")

    all_bytes = bits_to_bytes(_parity_decode(data, ws_start,
total_size * 8))

    if all_bytes[2:6] != b'STEG':
        raise ValueError("Failed to parse stego header: wrong
key or not a stego video")

    pos = 6
    flags = all_bytes[pos]; pos += 1
    pos += 1
    payload_len = struct.unpack('>I', all_bytes[pos:pos +
4])[0]; pos += 4
    fname_len = struct.unpack('>H', all_bytes[pos:pos +
2])[0]; pos += 2
    filename = all_bytes[pos:pos + fname_len].decode('utf-8',
errors='replace'); pos += fname_len
    payload = all_bytes[pos:pos + payload_len]

    return payload, bool(flags & 0x01), filename, bool(flags
& 0x02)
```

Mengekstrak pesan yang tersimpan dalam file MP4 dengan membaca dan mengurai data dari workspace, termasuk flag, nama file, dan payload.

GUI

Aplikasi dapat digunakan dalam bentuk GUI yang pembuatannya dibantu dengan library tkinter

histogram_window

Jendela baru untuk menampilkan histogram perbedaan nilai RGB video embed dan video cover asli

__init__

```
def __init__(self, parent, orig_path, stego_path):
        super().__init__(parent)
        self.title("Histogram Comparison")
        self.geometry("900x550")

        self.orig_path = orig_path
        self.stego_path = stego_path
        self._orig_frames = []
        self._stego_frames = []

        self._load_frames()

        ctrl = ttk.Frame(self)
        ctrl.pack(fill='x', padx=8, pady=4)
        ttk.Label(ctrl, text="Frame:").pack(side='left')
        self.frame_var = tk.IntVar(value=0)
        n = max(len(self._orig_frames) - 1, 0)
        self.spin = ttk.Spinbox(ctrl, from_=0, to=n,
textvariable=self.frame_var, width=6,
                                command=self._update_plot)
        self.spin.pack(side='left', padx=4)
        ttk.Button(ctrl, text="Refresh",
command=self._update_plot).pack(side='left', padx=4)

        self.fig = Figure(figsize=(9, 4.5), dpi=100)
        self.canvas = FigureCanvasTkAgg(self.fig,
master=self)
        self.canvas.get_tk_widget().pack(fill='both',
expand=True)
```

```
        self._update_plot()
```

Membuat GUI untuk menampilkan histogram RGB frame video cover asli dan video stego, serta menyediakan komponen untuk memilih dan membandingkan histogram tiap frame.

_load_frames

```
def _load_frames(self):
        for path, store in [(self.orig_path,
self._orig_frames),
                            (self.stego_path,
self._stego_frames)]:
            cap = cv2.VideoCapture(path)
            while True:
                ret, frame = cap.read()
                if not ret:
                    break
                store.append(frame)
            cap.release()
```

Membaca seluruh frame dari video asli dan video stego, lalu menyimpannya ke dalam list untuk kebutuhan analisis dan perbandingan histogram per frame.

_update_plot

```
 def _update_plot(self):
        idx = self.frame_var.get()
        if idx >= len(self._orig_frames) or idx >=
len(self._stego_frames):
            return

        orig = self._orig_frames[idx]
        stego = self._stego_frames[idx]

        self.fig.clear()
        colors = ['b', 'g', 'r']
        labels = ['Blue', 'Green', 'Red']

        for i, (color, label) in enumerate(zip(colors,
labels)):
```

```
            ax = self.fig.add_subplot(1, 3, i + 1)
            ax.hist(orig[:, :, i].ravel(), bins=256,
range=(0, 256),
                    alpha=0.5, color=color,
label=f'Original')
            ax.hist(stego[:, :, i].ravel(), bins=256,
range=(0, 256),
                    alpha=0.5, color='orange', label='Stego')
            ax.set_title(label)
            ax.legend(fontsize=7)
            ax.set_xlim(0, 256)

        self.fig.tight_layout()
        self.canvas.draw()
```

Memperbarui tampilan histogram per frame dengan membandingkan distribusi intensitas kanal Blue, Green, dan Red antara frame video cover (original) dan frame video stego, lalu menggambar ulang plot ke canvas.

extract_tab

Tab pada jendela program utama, menampilkan menu untuk mengekstrak pesan dari video steganografi

__init__

```
def __init__(self, parent):
        super().__init__(parent)
        self.stego_path = tk.StringVar()
        self.a51_key = tk.StringVar()
        self.stego_key = tk.StringVar()

        self._extracted_data = None
        self._extracted_filename = None
        self._extracted_hash = None

        self._build_ui()
```

Menginisialisasi tab GUI untuk proses ekstraksi dengan menyiapkan variabel input (path stego, key A5/1, stego-key), state hasil ekstraksi (data, filename, hash), lalu memanggil _build_ui untuk membangun seluruh komponen antarmuka.

_build_ui

```
def _build_ui(self):
        pad = {'padx': 6, 'pady': 3}

        row = 0
        ttk.Label(self, text="Video Format:").grid(row=row,
column=0, sticky='w', **pad)
        fmt_frame = ttk.Frame(self)
        fmt_frame.grid(row=row, column=1, sticky='w', **pad)
        self.video_format = tk.StringVar(value="avi")
        ttk.Radiobutton(fmt_frame, text="AVI",
variable=self.video_format, value="avi",

command=self._on_format_change).pack(side='left', padx=4)
        ttk.Radiobutton(fmt_frame, text="MP4",
variable=self.video_format, value="mp4",

command=self._on_format_change).pack(side='left', padx=4)

        row += 1
        ttk.Label(self, text="Stego Video:").grid(row=row,
column=0, sticky='w', **pad)
        ttk.Entry(self, textvariable=self.stego_path,
width=50).grid(row=row, column=1, sticky='ew', **pad)
        ttk.Button(self, text="Browse",
command=self._browse_stego).grid(row=row, column=2, **pad)

        row += 1
        ttk.Label(self, text="A5/1 Key (if
encrypted):").grid(row=row, column=0, sticky='w', **pad)
        ttk.Entry(self, textvariable=self.a51_key,
width=30).grid(row=row, column=1, sticky='w', **pad)

        row += 1
        ttk.Label(self, text="Stego-Key (if
random):").grid(row=row, column=0, sticky='w', **pad)
        self.skey_entry = ttk.Entry(self,
textvariable=self.stego_key, width=30)
```

```
        self.skey_entry.grid(row=row, column=1, sticky='w',
**pad)

        row += 1
        self.extract_btn = ttk.Button(self, text="Extract",
command=self._start_extract)
        self.extract_btn.grid(row=row, column=0,
columnspan=3, pady=12)

        row += 1
        self.status_var = tk.StringVar(value="Ready")
        ttk.Label(self, textvariable=self.status_var,
foreground='gray').grid(
            row=row, column=0, columnspan=3, sticky='w',
**pad)

        row += 1
        ttk.Label(self, text="Extracted
Message:").grid(row=row, column=0, sticky='nw', **pad)

        row += 1
        self.result_text = tk.Text(self, height=10, width=60,
state='disabled')
        self.result_text.grid(row=row, column=0,
columnspan=2, sticky='nsew', **pad)
        scrollbar = ttk.Scrollbar(self,
command=self.result_text.yview)
        scrollbar.grid(row=row, column=2, sticky='ns')
        self.result_text.config(yscrollcommand=scrollbar.set)

        row += 1
        self.hash_var = tk.StringVar()
        ttk.Label(self, textvariable=self.hash_var,
font=('Consolas', 9),
                  foreground='gray').grid(row=row, column=0,
columnspan=3, sticky='w', **pad)

        row += 1
        btn_frame = ttk.Frame(self)
```

```
        btn_frame.grid(row=row, column=0, columnspan=3,
pady=6)
        self.save_btn = ttk.Button(btn_frame, text="Save
Extracted File",
                                   command=self._save_file,
state='disabled')
        self.save_btn.pack(side='left', padx=8)
        self.verify_btn = ttk.Button(btn_frame, text="Verify
Integrity",

command=self._verify_integrity, state='disabled')
        self.verify_btn.pack(side='left', padx=8)

        self.columnconfigure(1, weight=1)
        self.rowconfigure(row - 2, weight=1)
        self._on_format_change()
```

Menyusun seluruh komponen antarmuka tab ekstraksi: pilihan format video, input path stego, input kunci A5/1 dan stego-key, tombol extract, area hasil ekstraksi, status, hash SHA-256, serta tombol simpan file dan verifikasi integritas.

_on_format_change

```
 def _on_format_change(self):
        if self.video_format.get() == "mp4":
            self.skey_entry.config(state='disabled')
        else:
            self.skey_entry.config(state='normal')
```

Menyesuaikan status input stego-key berdasarkan format video; untuk MP4 stego-key dinonaktifkan, untuk AVI diaktifkan.

_browse_stego

```
def _browse_stego(self):
        ext = self.video_format.get()
        path = filedialog.askopenfilename(
            filetypes=[(ext.upper(), f"*.{ext}")])
        if path:
            self.stego_path.set(path)
```

Membuka dialog pemilihan file untuk memilih video stego sesuai format, lalu menyimpan path terpilih.

_start_extract

```
def _start_extract(self):
        spath = self.stego_path.get()
        if not spath:
            messagebox.showwarning("Warning", "Select a stego
video first")
            return

        key = self.a51_key.get() or None
        skey = self.stego_key.get() or None

        self.extract_btn.config(state='disabled')
        self.status_var.set("Extracting...")
        self.hash_var.set("")

        def _do():
            try:
                data, is_file, fname = extract(spath,
a51_key=key, stego_key=skey)
                self.after(0, lambda:
self._extract_done(data, is_file, fname))
            except Exception as e:
                err_msg = str(e)
                self.after(0, lambda msg=err_msg:
self._extract_error(msg))

        threading.Thread(target=_do, daemon=True).start()
```

Memvalidasi input ekstraksi, menyiapkan key, mengubah status UI ke proses berjalan, lalu menjalankan ekstraksi di thread terpisah agar GUI tidak freeze.

_extract_done

```
def _extract_done(self, data, is_file, filename):
        self.extract_btn.config(state='normal')
        self.status_var.set("Extraction complete!")
```

```
        self._extracted_data = data
        self._extracted_filename = filename
        self._extracted_hash = sha256_digest(data)
        self.hash_var.set(f"SHA-256: {self._extracted_hash}")
        self.verify_btn.config(state='normal')

        if is_file:
            self.save_btn.config(state='normal')
            self.result_text.config(state='normal')
            self.result_text.delete('1.0', 'end')
            self.result_text.insert('1.0',
                f"[File extracted: {filename}]\nSize:
{len(data):,} bytes\n\n"
                "Click 'Save Extracted File' to save.\n"
                "Click 'Verify Integrity' to compare against
the original file.")
            self.result_text.config(state='disabled')
        else:
            self.save_btn.config(state='disabled')
            try:
                text = data.decode('utf-8')
            except Exception:
                text = data.decode('latin-1')
            self.result_text.config(state='normal')
            self.result_text.delete('1.0', 'end')
            self.result_text.insert('1.0', text)
            self.result_text.config(state='disabled')
```

Menangani hasil ekstraksi sukses, menyimpan data/metadata/hash, memperbarui tampilan hasil, dan mengaktifkan aksi lanjutan (simpan file atau verifikasi integritas sesuai tipe payload).

_extract_error

```
def _extract_error(self, msg):
        self.extract_btn.config(state='normal')
        self.status_var.set("Error")
        messagebox.showerror("Extract Error", msg)
```

Menangani kegagalan ekstraksi dengan mengembalikan status UI dan menampilkan pesan error.

_save_file

```
def _save_file(self):
        if self._extracted_data is None:
            return
        path = filedialog.asksaveasfilename(
            initialfile=self._extracted_filename or
"extracted")
        if path:
            with open(path, 'wb') as f:
                f.write(self._extracted_data)
            messagebox.showinfo("Saved", f"File saved to
{path}")
```

Menyimpan payload hasil ekstraksi ke lokasi yang dipilih pengguna.

_verify_integrity

```
def _verify_integrity(self):
        if self._extracted_hash is None:
            return
        path = filedialog.askopenfilename(title="Select
original file to compare")
        if not path:
            return
        with open(path, 'rb') as f:
            original_hash = sha256_digest(f.read())
        if original_hash == self._extracted_hash:
            messagebox.showinfo(
                "Integrity OK",
                f"SHA-256 match: payload is
intact.\n\n{self._extracted_hash}")
        else:
            messagebox.showerror(
                "Integrity FAIL",
                f"SHA-256 mismatch: payload may be
```

```
corrupted.\n\n"
                f"Original:  {original_hash}\nExtracted:
{self._extracted_hash}")
```

Memverifikasi integritas dengan membandingkan hash SHA-256 payload hasil ekstraksi dengan hash file asli yang dipilih pengguna.

embed_tab

Tab untuk menu menyisipkan pesan pada video cover pilihan

__init__

```
def __init__(self, parent):
        super().__init__(parent)
        self.cover_path = tk.StringVar()
        self.msg_type = tk.StringVar(value="text")
        self.file_path = tk.StringVar()
        self.scheme = tk.StringVar(value="3-3-2")
        self.use_encrypt = tk.BooleanVar(value=False)
        self.a51_key = tk.StringVar()
        self.insert_mode = tk.StringVar(value="sequential")
        self.stego_key = tk.StringVar()
        self.output_name = tk.StringVar()

        self._cover_video_path = None
        self._out_path = None
        self._last_mse = None
        self._last_psnr = None

        self._build_ui()
```

Menginisialisasi tab GUI untuk proses embedding dengan menyiapkan variabel input lalu memanggil _build_ui untuk membangun seluruh komponen antarmuka.

_build_ui

```
def _build_ui(self):
        pad = {'padx': 6, 'pady': 3}

        row = 0
        ttk.Label(self, text="Video Format:").grid(row=row,
```

```
column=0, sticky='w', **pad)
        fmt_frame = ttk.Frame(self)
        fmt_frame.grid(row=row, column=1, sticky='w', **pad)
        self.video_format = tk.StringVar(value="avi")
        ttk.Radiobutton(fmt_frame, text="AVI",
variable=self.video_format, value="avi",

command=self._on_format_change).pack(side='left', padx=4)
        ttk.Radiobutton(fmt_frame, text="MP4",
variable=self.video_format, value="mp4",

command=self._on_format_change).pack(side='left', padx=4)

        # Cover video
        row += 1
        ttk.Label(self, text="Cover Video:").grid(row=row,
column=0, sticky='w', **pad)
        ttk.Entry(self, textvariable=self.cover_path,
width=50).grid(row=row, column=1, sticky='ew', **pad)
        ttk.Button(self, text="Browse",
command=self._browse_cover).grid(row=row, column=2, **pad)

        # Message type
        row += 1
        ttk.Label(self, text="Message Type:").grid(row=row,
column=0, sticky='w', **pad)
        type_frame = ttk.Frame(self)
        type_frame.grid(row=row, column=1, sticky='w', **pad)
        ttk.Radiobutton(type_frame, text="Text",
variable=self.msg_type, value="text",

command=self._toggle_msg_type).pack(side='left', padx=4)
        ttk.Radiobutton(type_frame, text="File",
variable=self.msg_type, value="file",

command=self._toggle_msg_type).pack(side='left', padx=4)

        # Text input
        row += 1
```

```
        self.text_label = ttk.Label(self, text="Message:")
        self.text_label.grid(row=row, column=0, sticky='nw',
**pad)
        self.text_area = tk.Text(self, height=5, width=50)
        self.text_area.grid(row=row, column=1, columnspan=2,
sticky='ew', **pad)

        # File input
        row += 1
        self.file_label = ttk.Label(self, text="File:")
        self.file_entry = ttk.Entry(self,
textvariable=self.file_path, width=50)
        self.file_btn = ttk.Button(self, text="Browse",
command=self._browse_file)
        self.file_row = row

        # Scheme
        row += 1
        ttk.Label(self, text="LSB Scheme:").grid(row=row,
column=0, sticky='w', **pad)
        self.scheme_combo = ttk.Combobox(self,
textvariable=self.scheme,
                      values=["3-3-2", "4-2-2", "2-3-3"],
                      state='readonly', width=10)
        self.scheme_combo.grid(row=row, column=1, sticky='w',
**pad)

        # Encryption
        row += 1
        ttk.Checkbutton(self, text="Encrypt (A5/1)",
variable=self.use_encrypt,

command=self._toggle_encrypt).grid(row=row, column=0,
sticky='w', **pad)
        self.key_entry = ttk.Entry(self,
textvariable=self.a51_key, width=30, state='disabled')
        self.key_entry.grid(row=row, column=1, sticky='w',
**pad)
        self.key_label = ttk.Label(self, text="A5/1 Key")
```

```
        self.key_label.grid(row=row, column=2, sticky='w',
**pad)

        # Insertion mode
        row += 1
        ttk.Label(self, text="Insert Mode:").grid(row=row,
column=0, sticky='w', **pad)
        mode_frame = ttk.Frame(self)
        mode_frame.grid(row=row, column=1, sticky='w', **pad)
        self.seq_radio = ttk.Radiobutton(mode_frame,
text="Sequential", variable=self.insert_mode,
value="sequential",
                        command=self._toggle_random)
        self.seq_radio.pack(side='left', padx=4)
        self.rand_radio = ttk.Radiobutton(mode_frame,
text="Random", variable=self.insert_mode, value="random",
                        command=self._toggle_random)
        self.rand_radio.pack(side='left', padx=4)

        row += 1
        self.skey_label = ttk.Label(self, text="Stego-Key:")
        self.skey_label.grid(row=row, column=0, sticky='w',
**pad)
        self.skey_entry = ttk.Entry(self,
textvariable=self.stego_key, width=30, state='disabled')
        self.skey_entry.grid(row=row, column=1, sticky='w',
**pad)

        # Output
        row += 1
        ttk.Label(self, text="Output File:").grid(row=row,
column=0, sticky='w', **pad)
        ttk.Entry(self, textvariable=self.output_name,
width=50).grid(row=row, column=1, sticky='ew', **pad)
        ttk.Button(self, text="Browse",
command=self._browse_output).grid(row=row, column=2, **pad)

        # Buttons
        row += 1
```

```
        btn_frame = ttk.Frame(self)
        btn_frame.grid(row=row, column=0, columnspan=3,
pady=10)
        ttk.Button(btn_frame, text="Check Capacity",
command=self._check_capacity).pack(side='left', padx=8)
        self.embed_btn = ttk.Button(btn_frame, text="Embed",
command=self._start_embed)
        self.embed_btn.pack(side='left', padx=8)
        self.hist_btn = ttk.Button(btn_frame,
text="Histogram", command=self._show_histogram,
state='disabled')
        self.hist_btn.pack(side='left', padx=8)

        # Progress
        row += 1
        self.progress = ttk.Progressbar(self,
mode='determinate')
        self.progress.grid(row=row, column=0, columnspan=3,
sticky='ew', padx=8, pady=4)

        # Status
        row += 1
        self.status_var = tk.StringVar(value="Ready")
        ttk.Label(self, textvariable=self.status_var,
foreground='gray').grid(
            row=row, column=0, columnspan=3, sticky='w',
**pad)

        # Results
        row += 1
        self.result_var = tk.StringVar()
        ttk.Label(self, textvariable=self.result_var,
font=('Consolas', 10)).grid(
            row=row, column=0, columnspan=3, sticky='w',
**pad)

        self.columnconfigure(1, weight=1)
        self._toggle_msg_type()
        self._on_format_change()
```

Membangun seluruh komponen antarmuka tab Embed (input cover/payload, opsi skema, enkripsi, mode sisip, output, progress, status, dan hasil).

_on_format_change

```
def _on_format_change(self):
        if self.video_format.get() == "mp4":
            self.scheme_combo.config(state='disabled')
            self.seq_radio.config(state='disabled')
            self.rand_radio.config(state='disabled')
            self.skey_entry.config(state='disabled')
        else:
            self.scheme_combo.config(state='readonly')
            self.seq_radio.config(state='normal')
            self.rand_radio.config(state='normal')
            self._toggle_random()
```

Mengatur state komponen UI berdasarkan format video (MP4/AVI), termasuk enable/disable skema dan random stego-key.

_browse_cover

```
def _browse_cover(self):
        ext = self.video_format.get()
        path = filedialog.askopenfilename(
            filetypes=[(ext.upper(), f"*.{ext}")])
        if path:
            self.cover_path.set(path)
```

Membuka jendela untuk memilih path dari video cover yang ingin dipilih

_browse_file

```
def _browse_file(self):
        path = filedialog.askopenfilename()
        if path:
            self.file_path.set(path)
```

Membuka jendela untuk memilih path dari file yang ingin di sisipkan (payload)

browse_output

```
def _browse_output(self):
        ext = self.video_format.get()
```

```
        path = filedialog.asksaveasfilename(
            defaultextension=f".{ext}",
            filetypes=[(ext.upper(), f"*.{ext}")])
        if path:
            self.output_name.set(path)
```

Membuka dialog untuk memilih path dan nama file output hasil embedding.

_toggle_msg_type

```
def _toggle_msg_type(self):
        pad = {'padx': 6, 'pady': 3}
        if self.msg_type.get() == "text":
            self.text_label.grid()
            self.text_area.grid()
            self.file_label.grid_remove()
            self.file_entry.grid_remove()
            self.file_btn.grid_remove()
        else:
            self.text_label.grid_remove()
            self.text_area.grid_remove()
            self.file_label.grid(row=self.file_row, column=0,
sticky='w', **pad)
            self.file_entry.grid(row=self.file_row, column=1,
sticky='ew', **pad)
            self.file_btn.grid(row=self.file_row, column=2,
**pad)
```

Menyesuaikan tampilan input payload berdasarkan jenis pesan (text/file) dengan menampilkan komponen yang relevan.

_toggle_encrypt

```
def _toggle_encrypt(self):
        st = 'normal' if self.use_encrypt.get() else
'disabled'
        self.key_entry.config(state=st)
```

Mengatur enable/disable input key A5/1 sesuai status enkripsi.

_toggle_random

```
def _toggle_random(self):
        st = 'normal' if self.insert_mode.get() == 'random'
else 'disabled'
        self.skey_entry.config(state=st)
```

Mengatur enable/disable input stego-key sesuai mode penyisipan (random/sequential).

_check_capacity

```
def _check_capacity(self):
        cover = self.cover_path.get()
        if not cover:
            messagebox.showwarning("Warning", "Select a cover
video first")
            return
        try:
            is_mp4 = self.video_format.get() == 'mp4'
            cap = get_capacity(cover, self.scheme.get())
            if is_mp4:
                msg = (f"Max capacity: {cap:,} bytes
({cap/1024:.1f} KB)\n"
                       "Mode: MP4 container (parity encoding
in mdat)")
            else:
                msg = f"Max capacity: {cap:,} bytes
({cap/1024:.1f} KB)"

            payload_size = self._get_payload_size()
            if payload_size is not None:
                if payload_size <= cap:
                    msg += f"\nPayload size: {payload_size:,}
bytes ✓"
                else:
                    msg += f"\nPayload size: {payload_size:,}
bytes ✗"
            messagebox.showinfo("Capacity", msg)
        except Exception as e:
            messagebox.showerror("Error", str(e))
```

Menghitung kapasitas cover, membandingkan dengan ukuran payload, lalu menampilkan hasil.

_get_payload_size

```
def _get_payload_size(self):
        if self.msg_type.get() == "text":
            txt = self.text_area.get("1.0", "end-1c")
            if txt:
                return len(txt.encode('utf-8'))
        else:
            fp = self.file_path.get()
            if fp and os.path.isfile(fp):
                return os.path.getsize(fp)
        return None
```

Mengembalikan ukuran payload

_start_embed

```
def _start_embed(self):
        cover = self.cover_path.get()
        out = self.output_name.get()
        if not cover:
            messagebox.showwarning("Warning", "Select a cover
video")
            return
        if not out:
            messagebox.showwarning("Warning", "Specify output
file path")
            return

        is_file = self.msg_type.get() == "file"
        if is_file:
            fpath = self.file_path.get()
            if not fpath or not os.path.isfile(fpath):
                messagebox.showwarning("Warning", "Select a
valid file to embed")
                return
            with open(fpath, 'rb') as f:
```

```
                payload = f.read()
            filename = os.path.basename(fpath)
        else:
            txt = self.text_area.get("1.0", "end-1c")
            if not txt:
                messagebox.showwarning("Warning", "Enter a
message to embed")
                return
            payload = txt.encode('utf-8')
            filename = ""

        use_enc = self.use_encrypt.get()
        key = self.a51_key.get() if use_enc else None
        use_rand = self.insert_mode.get() == "random"
        skey = self.stego_key.get() if use_rand else None

        if use_enc and not key:
            messagebox.showwarning("Warning", "Enter an A5/1
encryption key")
            return
        if use_rand and not skey:
            messagebox.showwarning("Warning", "Enter a
stego-key for random insertion")
            return

        self._cover_video_path = cover
        self._out_path = out
        self.embed_btn.config(state='disabled')
        self.status_var.set("Embedding...")
        self.progress['value'] = 0
        self.result_var.set("")

        def _do():
            try:
                def progress(cur, total):
                    pct = int(cur / max(total, 1) * 100)
                    self.after(0, lambda:
self.progress.config(value=pct))
```

```
                avg_mse, avg_psnr, _, _ = embed(
                    cover, out, payload, self.scheme.get(),
is_file, filename,
                    use_enc, key, use_rand, skey,
progress_cb=progress)

                self._last_mse = avg_mse
                self._last_psnr = avg_psnr

                self.after(0, lambda:
self._embed_done(avg_mse, avg_psnr))
            except Exception as e:
                err_msg = str(e)
                self.after(0, lambda msg=err_msg:
self._embed_error(msg))

        threading.Thread(target=_do, daemon=True).start()
```

Memvalidasi semua input embedding, menyiapkan payload dan parameter (enkripsi/random), mengubah status UI, lalu menjalankan embedding di thread terpisah.

_embed_done

```
def _embed_done(self, mse, psnr, payload_hash):
        self.embed_btn.config(state='normal')
        self.hist_btn.config(state='normal')
        self.progress['value'] = 100
        self.status_var.set("Embedding complete!")
        if mse == 0.0 and psnr == float('inf'):
            quality = "MSE: 0.0  |  PSNR: ∞ dB  (MP4
container mode (frames unmodified))"
        else:
            psnr_str = f"{psnr:.2f}" if psnr != float('inf')
else "∞"
            quality = f"Avg MSE: {mse:.4f}  |  Avg PSNR:
{psnr_str} dB"
        self.result_var.set(f"{quality}\nSHA-256:
{payload_hash}")
```

Menangani hasil sukses embedding dengan update status/progress, mengaktifkan histogram, dan menampilkan metrik kualitas serta hash payload.
_embed_error
Menangani kegagalan embedding dengan reset state UI dasar (tombol/progress/status) dan menampilkan pesan error.
_show_histogram
Membuka jendela perbandingan histogram antara video cover dan video stego jika path keduanya tersedia.

```
def _embed_error(self, msg):
        self.embed_btn.config(state='normal')
        self.progress['value'] = 0
        self.status_var.set("Error")
        messagebox.showerror("Embed Error", msg)
```

```
def _show_histogram(self):
        if self._cover_video_path and self._out_path:
            HistogramWindow(self, self._cover_video_path,
self._out_path)
```

app

__init__

```
def __init__(self):
        super().__init__()
        self.title("LSB Steganography")
        self.geometry("780x680")
        self.minsize(700, 600)

        icon_path = Path(__file__).resolve().parent.parent /
"logo.ico"
        if icon_path.exists():
            self.iconbitmap(default=str(icon_path))

        style = ttk.Style()
        style.theme_use('clam')
```

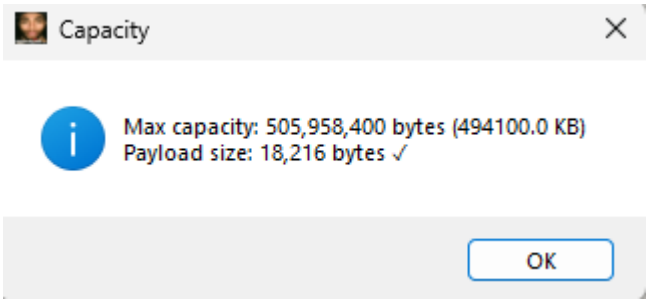
```
        notebook = ttk.Notebook(self)
        notebook.pack(fill='both', expand=True, padx=8,
pady=8)

        self.embed_tab = EmbedTab(notebook)
        self.extract_tab = ExtractTab(notebook)

        notebook.add(self.embed_tab, text="  Embed  ")
        notebook.add(self.extract_tab, text="  Extract  ")
```

Menginisialisasi tampilan utama aplikasi, mencakup pengaturan judul, ukuran window, ikon, tema, serta menyusun dua tab utama(embed dan extract)

Pengujian Program dan Analisis Hasil

1. Kombinasi Konfigurasi (AVI)

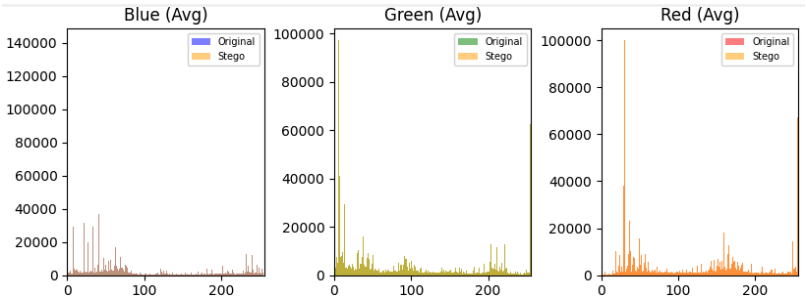
Pada pengujian bagian ini, semua konfigurasi akan menggunakan *sample* video format AVI berukuran 2.348 KB yang disisipkan file .txt berukuran 18 KB yang menunjukkan *capacity* dan *payload* dengan ukuran berikut:



Gambar 3.1 Capacity dan payload pengujian bagian 1

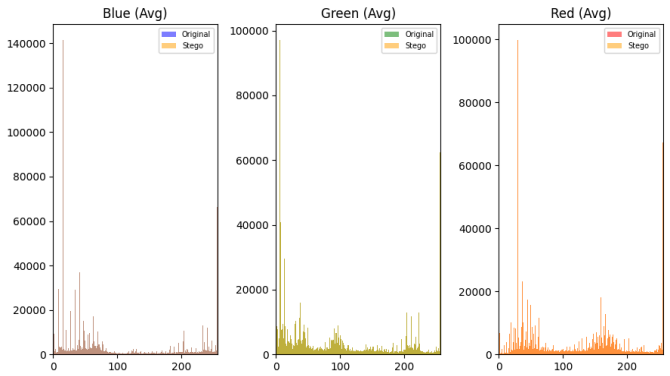
Selain itu, semua stego-key yang digunakan untuk mode *random* adalah "vestia" dan A5/1 *key* yang digunakan untuk mode enkripsi adalah "zeta"

1.1. Konfigurasi 1.1

Scheme	***Mode***	***Enkripsi***
3-3-2	Sequential	Tidak
PSNR & MSE	**Cek Integritas (SHA-256)**	
Avg MSE: 0.2692 | Avg PSNR: 53.83 dB	SHA-256 match: payload is intact. 12ec997b6dd046777383dff831be1562e065d94ad34120072c6ed1c5ae198c3c	
Histogram		
		

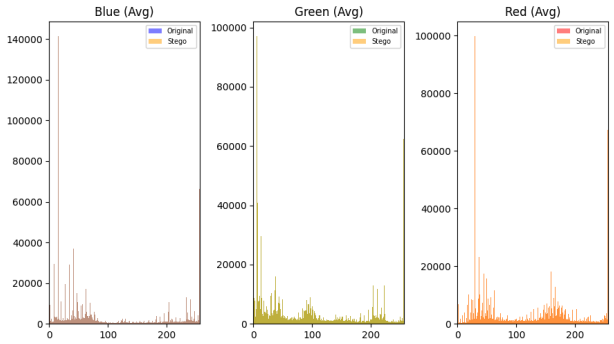
1.2. Konfigurasi 1.2

Scheme	*Mode*	*Enkripsi*
3-3-2	Sequential	A5/1
PSNR & MSE	**Cek Integritas (SHA-256)**	
Avg MSE: 0.2692 | Avg PSNR: 53.83 dB	SHA-256 match: payload is intact. 12ec997b6dd046777383dff831be1562e065d94ad34120072c6ed1c5ae198c3c	
Histogram		



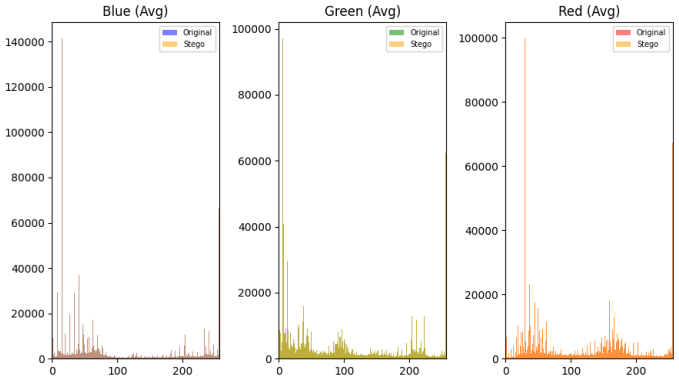
1.3. Konfigurasi 1.3

Scheme	*Mode*	*Enkripsi*
3-3-2	Random	Tidak
PSNR & MSE	**Cek Integritas (SHA-256)**	
Avg MSE: 0.1580 | Avg PSNR: 56.14 dB	SHA-256 match: payload is intact. 12ec997b6dd046777383dff831be1562e065d94ad34120072c6ed1c5ae198c3c	
Histogram		



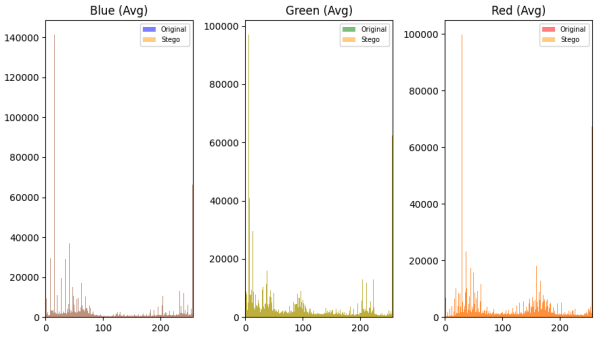
1.4. Konfigurasi 1.4

Scheme	*Mode*	*Enkripsi*
3-3-2	Random	A5/1
PSNR & MSE	**Cek Integritas (SHA-256)**	
Avg MSE: 0.1580 | Avg PSNR: 56.14 dB	SHA-256 match: payload is intact. 12ec997b6dd046777383dff831be1562e065d94ad34120072c6ed1c5ae198c3c	
Histogram		



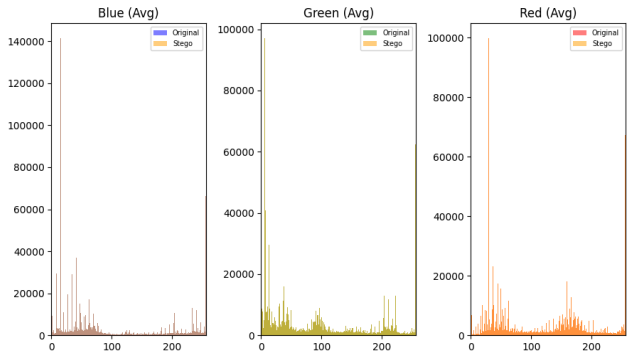
1.5. Konfigurasi 1.5

Scheme	*Mode*	*Enkripsi*
4-2-2	Sequential	Tidak
PSNR & MSE	**Cek Integritas (SHA-256)**	
Avg MSE: 0.5797 | Avg PSNR: 50.50 dB	SHA-256 match: payload is intact. 12ec997b6dd046777383dff831be1562e065d94ad34120072c6ed1c5ae198c3c	
Histogram		



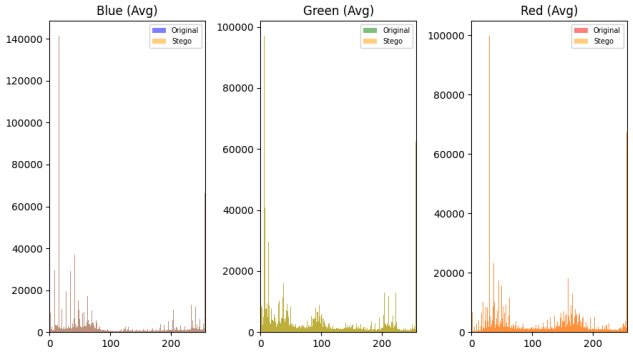
1.6. Konfigurasi 1.6

Scheme	*Mode*	*Enkripsi*
4-2-2	Sequential	A5/1
PSNR & MSE	**Cek Integritas (SHA-256)**	
Avg MSE: 0.5797 | Avg PSNR: 50.50 dB	SHA-256 match: payload is intact. 12ec997b6dd046777383dff831be1562e065d94ad34120072c6ed1c5ae198c3c	
Histogram		



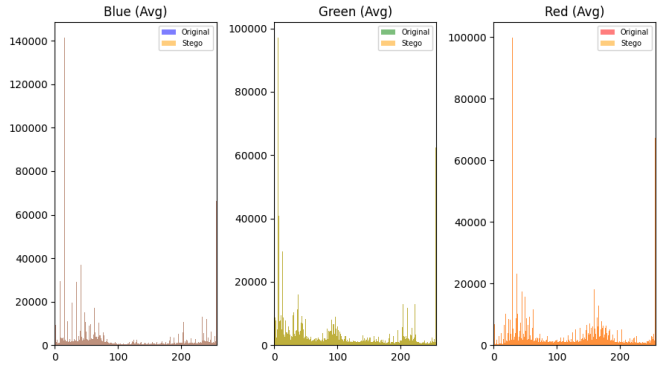
1.7. Konfigurasi 1.7

Scheme	*Mode*	*Enkripsi*
4-2-2	Random	Tidak
PSNR & MSE	**Cek Integritas (SHA-256)**	
Avg MSE: 0.3127 | Avg PSNR: 53.18 dB	SHA-256 match: payload is intact. 12ec997b6dd046777383dff831be1562e065d94ad34120072c6ed1c5ae198c3c	
Histogram		



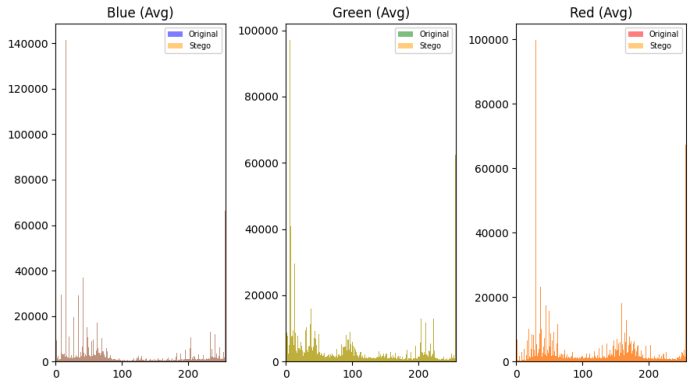
1.8. Konfigurasi 1.8

Scheme	*Mode*	*Enkripsi*
4-2-2	Random	A5/1
PSNR & MSE	**Cek Integritas (SHA-256)**	
Avg MSE: 0.3127 | Avg PSNR: 53.18 dB	SHA-256 match: payload is intact. 12ec997b6dd046777383dff831be1562e065d94ad34120072c6ed1c5ae198c3c	
Histogram		



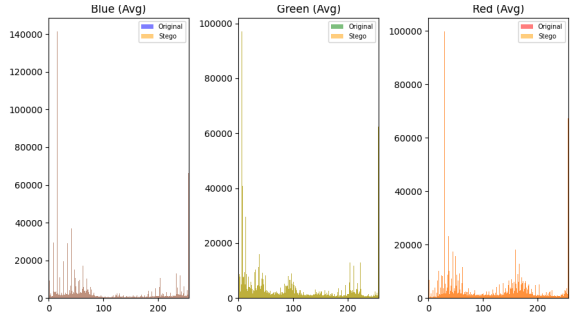
1.9. Konfigurasi 1.9

Scheme	*Mode*	*Enkripsi*
2-3-3	Sequential	Tidak
PSNR & MSE	**Cek Integritas (SHA-256)**	
Avg MSE: 0.2652 | Avg PSNR: 53.90 dB	SHA-256 match: payload is intact. 12ec997b6dd046777383dff831be1562e065d94ad34120072c6ed1c5ae198c3c	
Histogram		



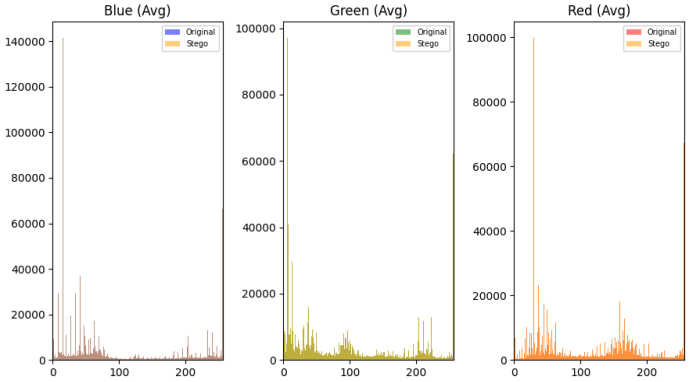
1.10. Konfigurasi 1.10

Scheme	*Mode*	*Enkripsi*
2-3-3	Sequential	A5/1
PSNR & MSE	**Cek Integritas (SHA-256)**	
Avg MSE: 0.2652 | Avg PSNR: 53.90 dB	SHA-256 match: payload is intact. 12ec997b6dd046777383dff831be1562e065d94ad34120072c6ed1c5ae198c3c	
Histogram		



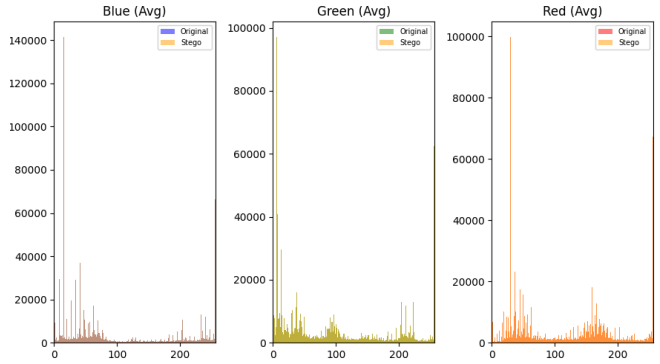
1.11. Konfigurasi 1.11

Scheme	*Mode*	*Enkripsi*
2-3-3	Random	Tidak
PSNR & MSE	**Cek Integritas (SHA-256)**	
Avg MSE: 0.1602 | Avg PSNR: 56.08 dB	SHA-256 match: payload is intact. 12ec997b6dd046777383dff831be1562e065d94ad34120072c6ed1c5ae198c3c	
Histogram		



1.12. Konfigurasi 1.12

Scheme	*Mode*	*Enkripsi*
2-3-3	Random	A5/1
PSNR & MSE	**Cek Integritas (SHA-256)**	
Avg MSE: 0.1602 | Avg PSNR: 56.08 dB	SHA-256 match: payload is intact. 12ec997b6dd046777383dff831be1562e065d94ad34120072c6ed1c5ae198c3c	
Histogram		



1.13. Analisis Hasil

Dari semua kasus pengujian di atas, didapatkan file-file sebagai berikut:

1.1.avi	31/03/2026 21:03	AVI Video File (VLC)	94.972 KB
1.2.avi	31/03/2026 21:11	AVI Video File (VLC)	94.972 KB
1.3.avi	31/03/2026 21:32	AVI Video File (VLC)	95.079 KB
1.4.avi	31/03/2026 21:24	AVI Video File (VLC)	95.079 KB
1.5.avi	31/03/2026 21:06	AVI Video File (VLC)	94.970 KB
1.6.avi	31/03/2026 21:15	AVI Video File (VLC)	94.971 KB
1.7.avi	31/03/2026 21:30	AVI Video File (VLC)	95.080 KB
1.8.avi	31/03/2026 21:27	AVI Video File (VLC)	95.080 KB
1.9.avi	31/03/2026 21:10	AVI Video File (VLC)	94.972 KB
1.10.avi	31/03/2026 21:16	AVI Video File (VLC)	94.971 KB
1.11.avi	31/03/2026 21:29	AVI Video File (VLC)	95.078 KB
1.12.avi	31/03/2026 21:28	AVI Video File (VLC)	95.078 KB
sample.avi	31/03/2026 18:04	AVI Video File (VLC)	2.348 KB

Gambar 3.2 Filesize setiap hasil steganografi

Berdasarkan hasil pengujian pada seluruh konfigurasi 1.1 hingga 1.12 , terlihat bahwa variasi skema bit (3-3-2, 4-2-2, 2-3-3), mode penyisipan (*sequential* dan *random*), serta penggunaan enkripsi A5/1 tidak memberikan perbedaan yang signifikan terhadap ukuran file hasil steganografi. Hal ini menunjukkan bahwa

metode LSB yang digunakan bekerja dengan memodifikasi bit pada piksel video secara konsisten sesuai kapasitas yang dialokasikan, bukan berdasarkan isi data.

Dari sisi kualitas, nilai PSNR dan MSE cenderung berada pada rentang yang relatif mirip antar konfigurasi. Hal ini mengindikasikan bahwa perubahan distribusi bit tidak terlalu mempengaruhi kualitas visual video secara signifikan, karena perubahan hanya terjadi pada bit-bit rendah (least significant bit).

Penggunaan mode *random* hanya mempengaruhi pola penyebaran bit yang dimodifikasi, bukan jumlahnya. Oleh karena itu, perbedaan kualitas maupun ukuran file tetap tidak signifikan dibandingkan mode sequential. Sementara itu, penggunaan enkripsi A5/1 tidak mempengaruhi PSNR, MSE, maupun ukuran file, karena enkripsi hanya mengubah isi *payload* sebelum disisipkan, bukan proses *embedding*-nya.

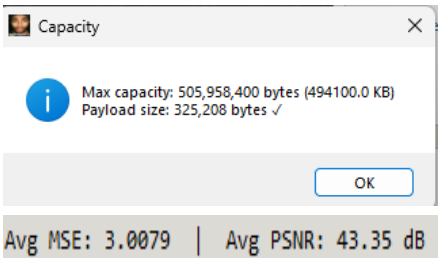
Dari hasil cek integritas menggunakan SHA-256, seluruh file hasil ekstraksi menunjukkan kesesuaian dengan file asli, yang berarti metode ini mampu mempertahankan keutuhan data dengan baik selama kapasitas tidak dilampaui.

Selain itu, berdasarkan pengamatan histogram, baik histogram rata-rata maupun histogram per frame tidak menunjukkan perbedaan yang signifikan antara video asli dan video hasil steganografi . Hal ini disebabkan karena metode LSB hanya memodifikasi bit-bit paling rendah dari setiap piksel, sehingga perubahan nilai intensitas piksel sangat kecil dan tidak cukup untuk menggeser distribusi histogram secara keseluruhan.

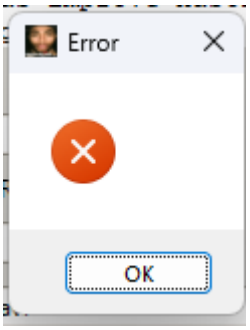
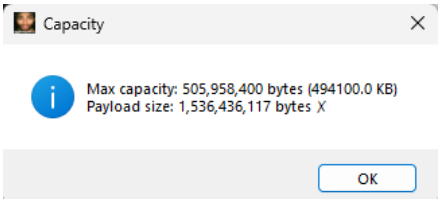
Pada histogram rata-rata, distribusi nilai piksel dari seluruh frame cenderung tetap stabil karena perubahan yang terjadi tersebar merata dan bersifat sangat kecil. Sementara itu, pada histogram per frame, meskipun perubahan terjadi di setiap frame, perubahannya tetap tidak cukup besar untuk menghasilkan perbedaan visual yang jelas.

2. Melebihi Kapasitas Sisip

2.1. Konfigurasi 2.1

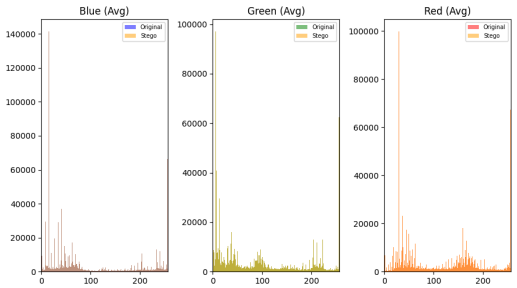
Melebihi batas sisip	***Jenis berkas***
Tidak	png (318 KB)
Cek kapasitas	**Analisis**
	Media cover AVI yang digunakan dapat menerima kapasitas sisip sampai 500 MB. Oleh karena itu, semua berkas di bawah ukuran tersebut dapat disisipkan. Hasil ekstraksi juga menunjukkan bahwa file yang diperoleh identik dengan file asli, yang dibuktikan melalui kecocokan nilai SHA-256 seperti hasil yang diperlihatkan di pengujian sebelumnya.

2.2. Konfigurasi 2.2

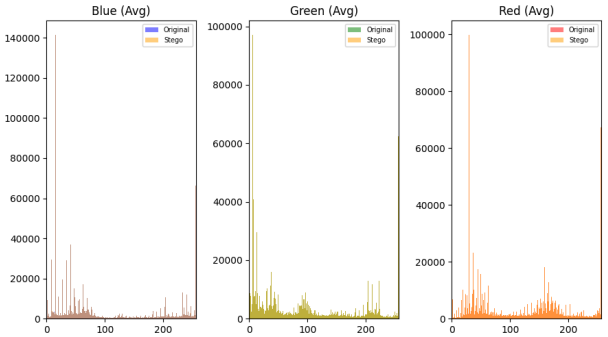
Melebihi batas sisip	***Jenis berkas***
Ya	7zip (1.54 GB)
Cek kapasitas	**Analisis**
 	Pada kali ini ketika mencoba file yang disisipkan jauh melebihi kapasitas maksimum media cover, proses *embedding* tidak dapat dilakukan dan proses akan berhenti, seperti gambar *error* di samping Hal ini karena LSB memiliki keterbatasan kapasitas yang bergantung pada jumlah piksel/frame yang tersedia.

3. Berbagai Jenis Berkas

3.1. Konfigurasi 3.1

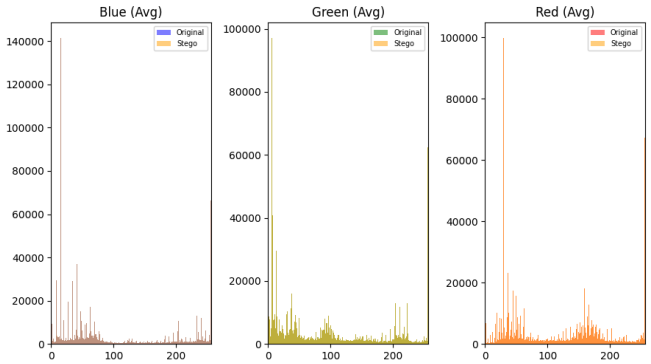
Scheme	*Berkas*	*Mode*	*Enkripsi*
3-3-2	pdf (236 KB)	Sequential	Tidak
PSNR & MSE		**Cek Integritas (SHA-256)**	
Avg MSE: 2.3956 | Avg PSNR: 44.34 dB		SHA-256 match: payload is intact. 9fa8569dcfcb1765df3d374ba219418ce2b1131b81ca6fb8f9113f28ea86e6ce	
Histogram			
			

3.2. Konfigurasi 3.2

Scheme	*Jenis berkas*	*Mode*	*Enkripsi*
3-3-2	docx (680 KB)	Sequential	Tidak
PSNR & MSE		**Cek Integritas (SHA-256)**	
Avg MSE: 6.0886 | Avg PSNR: 40.29 dB		SHA-256 match: payload is intact. 0125bf08d227269cdbc9b90f676bda61fe70aa03e9a5b61741e8ddf8b3ddd6af	
Histogram			
			

3.3. Konfigurasi 3.3

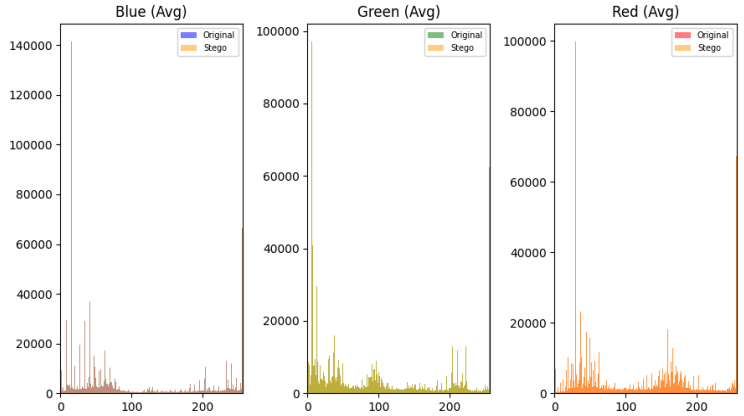
Scheme	*Jenis berkas*	*Mode*	*Enkripsi*
3-3-2	jpg (182 KB)	Sequential	Tidak
PSNR & MSE		**Cek Integritas (SHA-256)**	
Avg MSE: 1.8468 | Avg PSNR: 45.47 dB		SHA-256 match: payload is intact. 1c94914d359824484112a0bc3a2a2d00d83988914b41b798c445dba34a2580aa	
Histogram			



3.4. Konfigurasi 3.4

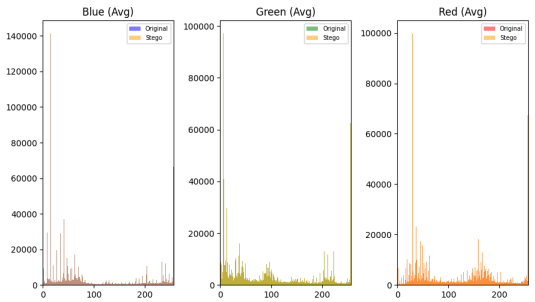
Scheme	*Jenis berkas*	*Mode*	*Enkripsi*
3-3-2	png (318 KB)	Sequential	Tidak
PSNR & MSE		**Cek Integritas (SHA-256)**	
Avg MSE: 3.0079 | Avg PSNR: 43.35 dB		SHA-256 match: payload is intact. 9c6f792fdc73540cd4e3411272942af41b0b2fe536a9d5eaee9071a57ab58f70	
Histogram			

Scheme	*Jenis berkas*	*Mode*	*Enkripsi*
3-3-2	png (318 KB)	Sequential	Tidak
PSNR & MSE		**Cek Integritas (SHA-256)**	
Avg MSE: 3.0079 | Avg PSNR: 43.35 dB		SHA-256 match: payload is intact. 9c6f792fdc73540cd4e3411272942af41b0b2fe536a9d5eaee9071a57ab58f70	



3.5. Konfigurasi 3.5

Scheme	*Berkas*	*Mode*	*Enkripsi*
3-3-2	exe (3731 KB)	Sequential	Tidak
PSNR & MSE		**Cek Integritas (SHA-256)**	
Avg MSE: 6.8026 | Avg PSNR: 40.54 dB		SHA-256 match: payload is intact. cfca124c2141b4171cebad48cc286751910b408aec69767db2bc3af38f277a06	
Histogram			



3.6. Analisis Hasil

Berdasarkan hasil pengujian berbagai jenis file (pdf, docx, jpg, png, exe) , metode steganografi yang digunakan tidak bergantung pada tipe file, melainkan hanya

pada ukuran file (*payload*). Semua jenis file dapat disisipkan selama ukurannya tidak melebihi kapasitas.

Nilai PSNR dan MSE tetap relatif konsisten karena ukuran file yang disisipkan masih dalam batas wajar. jenis file tidak mempengaruhi distribusi piksel secara langsung.

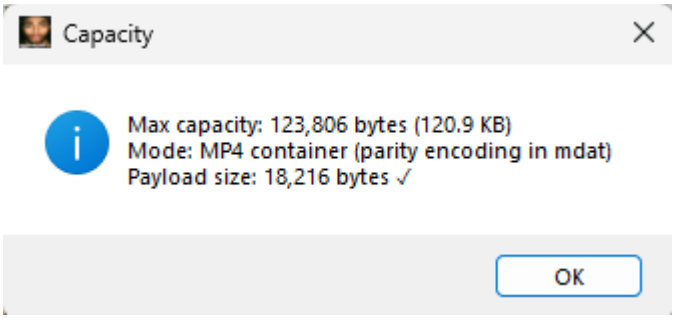
Hasil cek integritas SHA-256 menunjukkan bahwa semua file dapat diekstraksi kembali dengan sempurna.

Dari sisi histogram, histogram rata-rata tidak menunjukkan perbedaan yang terlihat antara video asli dan video hasil steganografi . Hal ini terjadi karena perubahan akibat penyisipan data tersebar secara merata pada seluruh frame, sehingga ketika dirata-ratakan, distribusi piksel tetap terlihat serupa dengan video asli.

Namun, pada histogram per frame, mulai terlihat adanya perbedaan yang lumayan mencolok Hal ini disebabkan karena setiap frame mengalami modifikasi bit yang berbeda-beda tergantung pada bagian payload yang disisipkan pada frame tersebut. Variasi lokal ini tidak cukup besar untuk mempengaruhi histogram rata-rata, tetapi dapat terlihat ketika dianalisis per frame.

4. Kombinasi Konfigurasi (MP4)

Pada pengujian bagian ini, semua konfigurasi akan menggunakan *sample* video format MP4 berukuran 2921 KB yang disisipkan file .txt berukuran 18 KB yang menunjukkan *capacity* dan *payload* dengan ukuran berikut:



Gambar 3.3 Capacity dan payload pengujian bagian 4

Selain itu, semua A5/1 *key* yang digunakan untuk mode enkripsi adalah “zeta”

4.1. Konfigurasi 4.1

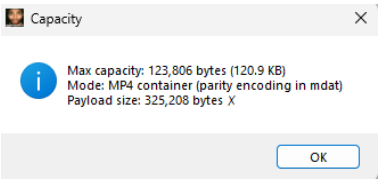
Melebihi batas sisip	*Enkripsi*	*Jenis berkas*
Tidak	Tidak	txt (18 KB)
PSNR & MSE		
`MSE: 0.0 | PSNR: ∞ dB (MP4 container mode (frames unmodified))`		
Cek Integritas (SHA-256)	**Analisis**	
SHA-256 match: payload is intact. 12ec997b6dd046777383dff831be1562e065d94ad34120072c6ed1c5ae198c3c	Nilai MSE yang diperoleh pada konfigurasi ini adalah 0 dan PSNR bernilai tak hingga (infinite) . Hal ini menunjukkan bahwa tidak terdapat perbedaan sama sekali antara frame video asli dan frame video hasil steganografi. Hal ini terjadi karena pada format MP4 proses penyisipan data tidak dilakukan dengan memodifikasi nilai piksel pada setiap frame seperti pada AVI, melainkan dilakukan pada bagian data stream video, khususnya pada atom *mdat* (media data). Dengan demikian, nilai piksel pada frame tidak mengalami perubahan. Integritas data juga tetap terjaga, dibuktikan dengan kecocokan hash SHA-256 antara file asli dan hasil ekstraksi.	

4.2. Konfigurasi 4.2

Melebihi batas sisip	*Enkripsi*	*Jenis berkas*
Tidak	A5/1	txt (18 KB)
PSNR & MSE		
`MSE: 0.0 | PSNR: ∞ dB (MP4 container mode (frames unmodified))`		
Cek Integritas (SHA-256)	**Analisis**	

Melebihi batas sisip	*Enkripsi*	*Jenis berkas*
Tidak	A5/1	txt (18 KB)
PSNR & MSE		
MSE: 0.0 | PSNR: ∞ dB (MP4 container mode (frames unmodified))		
SHA-256 match: payload is intact. 12ec997b6dd046777383dff831be1562e065d94ad34120072c6ed1c5ae198c3c	Sama seperti kasus 4.1, penggunaan enkripsi tetap membuat integritas data tetap terjaga, dibuktikan dengan kecocokan hash SHA-256 antara file asli dan hasil ekstraksi.	

4.3. Konfigurasi 4.3

Melebihi batas sisip	*Enkripsi*	*Jenis berkas*
Ya	-	png (318 KB)
Cek kapasitas		
		
Analisis		
Kapasitas penyisipan pada file MP4 lebih terbatas dibandingkan dengan AVI . Hal ini disebabkan karena metode penyisipan pada MP4 tidak memanfaatkan seluruh piksel frame, melainkan hanya menggunakan bagian tertentu dari struktur file, yaitu atom *mdat*. Akibatnya, kapasitas maksimum yang tersedia untuk embedding pada MP4 menjadi lebih kecil. Ketika ukuran payload melebihi kapasitas tersebut, proses penyisipan tidak dapat dilakukan.		

Kesimpulan

Berdasarkan hasil implementasi dan pengujian yang telah dilakukan, program steganografi video berbasis metode LSB (Least Significant Bit) dapat direalisasikan dengan dukungan tiga skema distribusi bit (3-3-2, 4-2-2, dan 2-3-3), dua mode penyisipan (sequential dan random), serta penambahan enkripsi A5/1. Selain itu, program juga mampu menangani dua format video, yaitu AVI sebagai format utama dan MP4 sebagai pengembangan tambahan. Proses penyisipan dan ekstraksi data berjalan dengan baik pada berbagai skenario pengujian, yang ditunjukkan melalui kesesuaian nilai hash SHA-256 antara berkas asli dan hasil ekstraksi.

Pada pengujian format AVI dengan 12 variasi konfigurasi, perubahan parameter seperti skema bit, mode penyisipan, maupun penggunaan enkripsi tidak menunjukkan pengaruh yang signifikan terhadap kualitas visual maupun ukuran berkas keluaran. Nilai PSNR yang relatif tinggi pada seluruh percobaan mengindikasikan bahwa perubahan akibat penyisipan data berada pada tingkat yang sangat kecil dan tidak terdeteksi secara visual. Hal ini sejalan dengan karakteristik metode LSB yang hanya memodifikasi bit dengan bobot paling rendah pada piksel. Hasil analisis histogram, baik rata-rata maupun per frame, juga tidak memperlihatkan perbedaan distribusi piksel yang mencolok antara video asli dan video hasil penyisipan. Namun, semakin besar berkas yang disisipkan, akan semakin terlihat perbedaan pada histogram terutama pada histogram per-frame. Sedangkan, pada histogram rata-rata tetap tidak terlihat perbedaan yang mencolok.

Penggunaan enkripsi A5/1 tidak memberikan perubahan pada nilai PSNR, MSE, maupun ukuran file karena proses enkripsi dilakukan pada data sebelum disisipkan. Sementara itu, mode random hanya memengaruhi posisi penyisipan bit tanpa mengubah jumlah bit yang dimodifikasi. Oleh karena itu, kedua aspek tersebut tidak berdampak langsung pada karakteristik visual maupun ukuran file, melainkan lebih berperan dalam meningkatkan aspek keamanan data yang disisipkan.

Pada pengujian kapasitas, sistem mampu mendeteksi kondisi ketika ukuran payload melebihi kapasitas media dan memberikan respons berupa pesan kesalahan sehingga proses tidak dilanjutkan. Batasan ini berkaitan langsung dengan jumlah piksel yang tersedia pada video sebagai media penyisipan. Pengujian terhadap berbagai jenis berkas seperti pdf, docx, jpg, png, dan exe menunjukkan bahwa metode yang digunakan tidak bergantung pada tipe file, melainkan pada ukuran data yang akan disisipkan.

Untuk format MP4, mekanisme penyisipan dilakukan pada level kontainer, yaitu dengan memanfaatkan byte pada blok mdat tanpa memodifikasi nilai piksel pada frame video. Kondisi ini menyebabkan nilai MSE yang diperoleh sebesar 0 dan PSNR bernilai tak hingga, karena tidak terjadi perubahan pada komponen visual video. Namun demikian, kapasitas penyisipan pada MP4 relatif lebih terbatas dibandingkan AVI karena hanya sebagian data dalam blok mdat yang digunakan, dan setiap bit pesan memerlukan satu byte. Selain itu, metode ini tidak mendukung mode random karena penyisipan dilakukan secara sekuensial mengikuti struktur kontainer, sehingga aspek keamanan tetap bergantung pada penggunaan enkripsi.

Secara keseluruhan, implementasi program telah memenuhi kebutuhan fungsional yang dirancang. Metode LSB menunjukkan kinerja yang baik dalam menjaga kualitas visual video serta integritas data yang disisipkan. Perbedaan pendekatan antara format AVI dan MP4 terletak pada lokasi penyisipan data, yaitu pada level piksel untuk AVI dan pada

level kontainer untuk MP4, yang masing-masing memiliki konsekuensi terhadap kapasitas dan karakteristik hasil penyisipan.

Daftar Pustaka

- Nados, Wessam & Rasheed, Nada. (1999). Hiding Text in AVI Video File by Method of Least Significant Bit. Journal of Engineering and Applied Sciences. 14. 10.36478/jeasci.2019.1999.2006.
- Dom Gómez, J. (GitHub Repository). VideoStego - Video Steganography. Diakses dari: https://github.com/JavDomGom/videostego
- Kavitha, R., & Murugan, A. (2007). "Lossless Steganography on AVI File using Swapping Algorithm." Proceedings of International Conference on Computational Intelligence and Multimedia Applications 2007. IEEE. DOI: 10.1109/ICCIMA.2007.380

Lampiran

Repositori: https://github.com/aldoyfa/LSB-Steganography
Video Demo: https://bit.ly/DemoKripto16

Pembagian Tugas:

NIM	Nama	Pembagian Tugas
18223041	Luckman F Arvasirri	- Algoritma LSB
18223113	Aldoy Fauzan Avanza	- Algoritma Enkripsi A51 - GUI Embed tab dan Extract tab
18223130	M Rabbani K A	- Utils (fungsi-fungsi pembantu) - GUI Histogram

Pernyataan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.

Luckman Fakhmanidris Arvasirri 18223041	Aldoy Fauzan Avanza 18223113	M Rabbani K A 18223130